

ĐẠI HỌC CÔNG NGHIỆP HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

TIỂU LUẬN CƠ SỞ DỮ LIỆU NÂNG CAO
KẾT THÚC HỌC PHẦN

ĐỀ TÀI

XÂY DỰNG VÀ QUẢN TRỊ CƠ SỞ DỮ LIỆU QUẢN LÝ
THIẾT BỊ MÁY TÍNH VỚI CƠ CHẾ SAO CHÉP VÀ
PHÂN MẢNH DỮ LIỆU TRÊN MONGODB

HỌC VIÊN: NGUYỄN HUY HOÀNG

MÃ HỌC VIÊN: 2026700148

LỚP: 20261IT7305002 - THẠC SĨ KHÓA 16 ĐỢT 2

NGÀNH: HỆ THỐNG THÔNG TIN

GIẢNG VIÊN: TS. NGUYỄN THỊ HỒNG LOAN

Hà Nội – Năm 2026

ĐẠI HỌC CÔNG NGHIỆP HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

TIỂU LUẬN CƠ SỞ DỮ LIỆU NÂNG CAO
KẾT THÚC HỌC PHẦN

ĐỀ TÀI

XÂY DỰNG VÀ QUẢN TRỊ CƠ SỞ DỮ LIỆU QUẢN LÝ
THIẾT BỊ MÁY TÍNH VỚI CƠ CHẾ SAO CHÉP VÀ
PHÂN MẢNH DỮ LIỆU TRÊN MONGODB

HỌC VIÊN: NGUYỄN HUY HOÀNG

MÃ HỌC VIÊN: 2026700148

LỚP: 20261IT7305002 - THẠC SĨ KHÓA 16 ĐỢT 2

NGÀNH: HỆ THỐNG THÔNG TIN

GIẢNG VIÊN: TS. NGUYỄN THỊ HỒNG LOAN

Hà Nội – Năm 2026

LỜI CẢM ƠN

Trước tiên, em xin gửi lời cảm ơn chân thành và sâu sắc tới TS. Nguyễn Thị Hồng Loan – giảng viên học phần Cơ sở dữ liệu nâng cao, Trường Công nghệ Thông tin và Truyền thông, Đại học Công nghiệp Hà Nội. Trong suốt quá trình học tập và thực hiện tiểu luận, giảng viên đã tận tình truyền đạt kiến thức, định hướng nội dung nghiên cứu và đưa ra những góp ý quý báu giúp em hoàn thiện báo cáo này.

Em cũng xin gửi lời cảm ơn tới các thầy cô Trường Công nghệ Thông tin và Truyền thông đã tạo điều kiện thuận lợi về môi trường học tập và nghiên cứu, cùng các anh chị, bạn bè trong lớp đã trao đổi, chia sẻ kinh nghiệm trong suốt quá trình học tập.

Mặc dù đã cố gắng hoàn thành tiểu luận với tất cả sự nỗ lực của bản thân, song do thời gian và kiến thức còn hạn chế nên báo cáo khó tránh khỏi những thiếu sót. Em rất mong nhận được những ý kiến đóng góp của giảng viên để em tiếp tục hoàn thiện kiến thức của mình.

Em xin chân thành cảm ơn!

Hà Nội, ngày tháng năm 2026

Học viên thực hiện

Nguyễn Huy Hoàng

MỤC LỤC

	Trang
LỜI CẢM ƠN	i
DANH MỤC CÁC THUẬT NGỮ, KÝ HIỆU VÀ CÁC CHỮ VIẾT TẮT	iv
DANH MỤC HÌNH VẼ.....	v
DANH MỤC BẢNG BIỂU	vi
MỞ ĐẦU.....	1
1. Lý do lựa chọn đề tài	1
2. Mục tiêu nghiên cứu	1
3. Đối tượng và phạm vi nghiên cứu	2
4. Kết quả mong muốn đạt được của đề tài	2
5. Cấu trúc báo cáo	2
CHƯƠNG 1. PHÂN TÍCH VÀ THIẾT KẾ CƠ SỞ DỮ LIỆU TRÊN MONGODB 3	3
1.1. Phân tích bài toán và dữ liệu nguồn.....	3
1.1.1. Mô tả bài toán và dữ liệu cần quản lý.....	3
1.1.2. Xác định các đối tượng và mối quan hệ.....	5
1.2. Phân tích và lựa chọn mô hình dữ liệu MongoDB	7
1.2.1. Phân tích khả năng sử dụng <i>Embedded Model</i>	7
1.2.2. Phân tích khả năng sử dụng <i>Reference/Normalized Model</i>	8
1.2.3. Lựa chọn và giải thích mô hình dữ liệu.....	9
1.3. Thiết kế cấu trúc cơ sở dữ liệu MongoDB	10
1.3.1. Xác định các <i>collection</i> và mối liên kết.....	10
1.3.2. Thiết kế <i>document</i> , <i>embedded document</i> , <i>array</i> và <i>reference</i>	11
1.3.3. Minh họa các <i>document</i> mẫu.....	12
1.4. Tạo lập và thao tác dữ liệu.....	12
1.4.1. Tạo <i>collection</i> và nhập dữ liệu	13
1.4.2. Thực hiện tìm kiếm và cập nhật dữ liệu	16
1.4.3. Thao tác với <i>embedded document</i> và <i>array</i>	18
Kết luận Chương 1	20
CHƯƠNG 2. KHAI THÁC VÀ TỔNG HỢP DỮ LIỆU TRÊN MONGODB	21
2.1. Xây dựng và thực hiện các yêu cầu truy vấn.....	21
2.1.1. Xác định các yêu cầu khai thác theo nghiệp vụ	21
2.1.2. Thực hiện và phân tích kết quả truy vấn	22
2.2. Tổng hợp dữ liệu bằng <i>Aggregation Pipeline</i>	23
2.2.1. Lọc và lựa chọn dữ liệu với <i>\$match</i> , <i>\$project</i>	24
2.2.2. Nhóm và tính toán với <i>\$group</i> , <i>\$sum</i> , <i>\$avg</i>	25
2.2.3. Sắp xếp và kết hợp các <i>stage</i> trong <i>Pipeline</i>	27

2.3. Kết hợp và đánh giá kết quả khai thác dữ liệu.....	29
2.3.1. Kết hợp các collection bằng \$lookup.....	29
2.3.2. Phân tích và đánh giá kết quả theo yêu cầu nghiệp vụ.....	31
Kết luận Chương 2.....	31
CHƯƠNG 3. TRIỂN KHAI CƠ SỞ DỮ LIỆU PHÂN TÁN TRONG MONGODB	
.....	32
3.1. Xây dựng và kiểm tra Replica Set.....	32
3.1.1. Cấu hình Replica Set.....	32
3.1.2. Kiểm tra Primary, Secondary và sao chép dữ liệu.....	33
3.2. Xây dựng Sharded Cluster.....	37
3.2.1. Phân tích và lựa chọn Shard Key.....	38
3.2.2. Cấu hình và thực hiện Sharding.....	40
3.3. Đánh giá kết quả triển khai.....	42
3.3.1. Kiểm tra sao chép dữ liệu.....	42
3.3.2. Kiểm tra phân bố dữ liệu trên các Shard.....	44
Kết luận Chương 3.....	48
KẾT LUẬN.....	49
Kết quả đạt được.....	49
Hạn chế.....	49
Hướng phát triển.....	49
TÀI LIỆU THAM KHẢO.....	50

DANH MỤC CÁC THUẬT NGỮ, KÝ HIỆU VÀ CÁC CHỮ VIẾT TẮT

Thuật ngữ / Viết tắt	Tiếng Anh	Ý nghĩa
Aggregation Pipeline	Aggregation Pipeline	Đường ống tổng hợp dữ liệu gồm nhiều giai đoạn (stage) xử lý nối tiếp
BSON	Binary JSON	Định dạng nhị phân MongoDB dùng để lưu trữ document
Chunk	Chunk	Khối dữ liệu ứng với một khoảng giá trị liên tục của khóa phân mảnh
Config server	Config Server	Máy chủ cấu hình, lưu siêu dữ liệu (metadata) của cụm phân mảnh
CRUD	Create, Read, Update, Delete	Các thao tác tạo, đọc, cập nhật và xóa dữ liệu
CSDL	–	Cơ sở dữ liệu
Document	Document	Tài liệu – đơn vị lưu trữ dữ liệu cơ bản của MongoDB
JSON	JavaScript Object Notation	Định dạng biểu diễn dữ liệu dạng văn bản
mongos	MongoDB Shard Router	Bộ định tuyến truy vấn trong cụm phân mảnh
mongosh	MongoDB Shell	Công cụ dòng lệnh tương tác với MongoDB
NSX	–	Nhà sản xuất
Oplog	Operations Log	Nhật ký thao tác dùng để sao chép dữ liệu trong Replica Set
Primary / Secondary	Primary / Secondary	Nút chính (nhận thao tác ghi) / nút phụ (sao chép dữ liệu)
Replica Set	Replica Set	Tập bản sao – nhóm tiến trình mongod cùng duy trì một tập dữ liệu
Shard	Shard	Phân mảnh – nút (Replica Set) lưu một phần dữ liệu của cụm
Shard key	Shard Key	Khóa phân mảnh, quyết định document thuộc chunk/shard nào
Sharded Cluster	Sharded Cluster	Cụm phân mảnh gồm các shard, config server và mongos
YC	–	Yêu cầu khai thác dữ liệu theo nghiệp vụ
Zone	Zone	Vùng – nhóm shard được gán một hoặc nhiều khoảng giá trị khóa phân mảnh

DANH MỤC HÌNH VẼ

Hình 1.1. Sơ đồ thực thể – mối quan hệ của bài toán quản lý thiết bị máy tính	6
Hình 1.2. Hai phương án mô hình nhúng (Embedded Model).....	8
Hình 1.3. Phương án mô hình tham chiếu (Reference Model)	9
Hình 1.4. Mô hình dữ liệu MongoDB được lựa chọn	11
Hình 1.5. Kết quả kiểm tra ràng buộc \$jsonSchema và chỉ mục duy nhất	16
Hình 1.6. Kết quả truy vấn mảng với \$elemMatch, \$size và \$all	19
Hình 2.1. Luồng xử lý Aggregation Pipeline của yêu cầu YC8.....	24
Hình 2.2. Kết quả YC8 – thống kê giá laptop theo nhà sản xuất	26
Hình 2.3. Kết quả YC14 – kết hợp makers và products bằng \$lookup.....	29
Hình 3.1. Kiến trúc Replica Set rs0 gồm ba nút.....	33
Hình 3.2. Trạng thái các thành viên của Replica Set rs0.....	34
Hình 3.3. Đọc dữ liệu và thử ghi tại nút Secondary mongo2.....	35
Hình 3.4. Dừng nút Primary mongo1 – Replica Set bầu mongo2 làm Primary mới	36
Hình 3.5. Khởi động lại mongo1 – nút tự đồng bộ và giành lại vai trò Primary	37
Hình 3.6. Kiến trúc Sharded Cluster được triển khai	38
Hình 3.7. Dừng nút Primary shard1a – cụm vẫn phục vụ đọc, ghi qua mongos.....	43
Hình 3.8. Kết quả getShardDistribution() của collection products	45
Hình 3.9. Phân bố document của collection products theo nhà sản xuất trên hai shard	46
Hình 3.10. Kế hoạch thực thi của truy vấn có và không có shard key.....	46

DANH MỤC BẢNG BIỂU

Bảng 1.1. Dữ liệu bảng PRODUCT (30 bản ghi).....	3
Bảng 1.2. Dữ liệu bảng LAPTOP (10 bản ghi)	4
Bảng 1.3. Dữ liệu bảng PRINTER (7 bản ghi).....	4
Bảng 1.4. Thống kê số sản phẩm theo nhà sản xuất và loại sản phẩm.....	4
Bảng 1.5. Các mẫu truy cập dữ liệu chính của nghiệp vụ	6
Bảng 1.6. So sánh các phương án mô hình dữ liệu	9
Bảng 1.7. Ảnh xạ thuộc tính nguồn sang cấu trúc MongoDB.....	11
Bảng 1.8. Cấu trúc document của collection products	12
Bảng 1.9. Môi trường thực nghiệm	13
Bảng 2.1. Danh sách yêu cầu khai thác dữ liệu theo nghiệp vụ	21
Bảng 2.2. Kết quả YC1 – laptop giá dưới 1.000 USD	22
Bảng 2.3. Kết quả YC3 – laptop màn hình từ 15,4 inch và RAM từ 1024 MB	23
Bảng 2.4. Kết quả YC5 – báo cáo cấu hình laptop.....	24
Bảng 2.5. Kết quả YC12 – phân khúc laptop theo giá	28
Bảng 3.1. Giá trị băm dữ liệu (dbHash) trên các nút của rs0	36
Bảng 3.2. Phân tích các phương án shard key cho collection products	39
Bảng 3.3. Các thành phần của Sharded Cluster.....	40
Bảng 3.4. Kết quả kiểm tra sao chép dữ liệu trên các nút của hai shard	43
Bảng 3.5. Kết quả định tuyến truy vấn trên Sharded Cluster	46
Bảng 3.6. So sánh hai phương án shard key qua thực nghiệm.....	47
Bảng 3.7. Đánh giá kết quả quản trị dữ liệu trên MongoDB.....	48

MỞ ĐẦU

1. Lý do lựa chọn đề tài

Khối lượng và mức độ đa dạng của dữ liệu tăng nhanh đặt ra yêu cầu cao đối với hệ quản trị cơ sở dữ liệu (CSDL) về tính linh hoạt của lược đồ, tính sẵn sàng và khả năng mở rộng. Với dữ liệu có cấu trúc không đồng nhất hoặc cần mở rộng theo chiều ngang, các hệ quản trị NoSQL – tiêu biểu là MongoDB – có nhiều ưu điểm nhờ mô hình dữ liệu hướng tài liệu (document) và các cơ chế phân tán tích hợp sẵn.

Bài toán quản lý thiết bị máy tính là ví dụ điển hình cho dữ liệu không đồng nhất: máy tính xách tay được mô tả bởi tốc độ, bộ nhớ, ổ cứng, màn hình, còn máy in được mô tả bởi khả năng in màu và công nghệ in. Mô hình quan hệ buộc phải tách nhiều bảng và nối bảng khi truy vấn, trong khi MongoDB biểu diễn tự nhiên bằng document và embedded document. Ngoài ra, hệ thống danh mục sản phẩm cần hoạt động liên tục và mở rộng khi dữ liệu tăng, nên sao chép (Replica Set) và phân mảnh (Sharding) là những nội dung quản trị then chốt.

Xuất phát từ những lý do trên, tiểu luận lựa chọn đề tài **“Xây dựng và quản trị cơ sở dữ liệu quản lý thiết bị máy tính với cơ chế sao chép và phân mảnh dữ liệu trên MongoDB”**. Đề tài giúp vận dụng tổng hợp kiến thức của học phần – từ mô hình hóa, truy vấn, tổng hợp dữ liệu đến CSDL phân tán – trên bộ dữ liệu kinh điển trích từ giáo trình [6].

2. Mục tiêu nghiên cứu

- Phân tích bài toán quản lý thiết bị máy tính từ các bảng dữ liệu PRODUCT, LAPTOP, PRINTER; xác định các đối tượng và mối quan hệ giữa sản phẩm, máy tính xách tay và máy in.
- Phân tích, so sánh mô hình nhúng (Embedded) và mô hình tham chiếu (Reference); lựa chọn và thiết kế mô hình dữ liệu MongoDB phù hợp, có giải thích.
- Tạo lập cơ sở dữ liệu, nhập đầy đủ và chính xác dữ liệu nguồn; thực hiện các thao tác truy vấn, cập nhật và tổng hợp dữ liệu theo nghiệp vụ.
- Cấu hình cơ chế sao chép dữ liệu (Replica Set), kiểm tra sự đồng bộ giữa các nút và khả năng chuyển đổi dự phòng.
- Phân tích phương án phân bố dữ liệu, lựa chọn khóa phân mảnh, cấu hình phân mảnh (Sharding) và kiểm tra sự phân bố dữ liệu trên các nút; từ đó đánh giá kết quả quản trị dữ liệu trên MongoDB.

3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu: mô hình dữ liệu hướng tài liệu của MongoDB (document, embedded document, array, reference); ngôn ngữ truy vấn MongoDB và Aggregation Pipeline; cơ chế sao chép dữ liệu bằng Replica Set và cơ chế phân mảnh dữ liệu bằng Sharded Cluster.

Phạm vi nghiên cứu: dữ liệu ba bảng PRODUCT (30 bản ghi), LAPTOP (10) và PRINTER (7) theo [6]; MongoDB Community Server 8.0 triển khai bằng Docker trên một máy tính cá nhân, mô phỏng cụm 13 tiến trình máy chủ. Chương 3 đi theo **hướng 2 – triển khai CSDL phân tán** (sao chép và phân mảnh) phù hợp yêu cầu đề tài; bảo mật, tối ưu chỉ mục chuyên sâu và giao tác đa tài liệu nằm ngoài phạm vi.

4. Kết quả mong muốn đạt được của đề tài

- Mô hình dữ liệu MongoDB được lựa chọn có lập luận rõ ràng; CSDL QLThietBi gồm hai collection products (30 document) và makers (8 document), có ràng buộc kiểm tra \$jsonSchema và chỉ mục duy nhất.
- Bộ 16 yêu cầu khai thác dữ liệu theo nghiệp vụ được hiện thực bằng truy vấn find, Aggregation Pipeline và \$lookup, kèm kết quả thực thi và phân tích.
- Replica Set ba nút hoạt động ổn định; kiểm chứng được sự đồng bộ dữ liệu giữa các nút và khả năng tự động bầu Primary mới khi có sự cố.
- Sharded Cluster gồm hai shard (mỗi shard là một Replica Set ba nút), Config Server Replica Set và bộ định tuyến mongos; dữ liệu được phân bố đúng theo khóa phân mảnh đã lựa chọn.

5. Cấu trúc báo cáo

Ngoài phần Mở đầu, Kết luận và Tài liệu tham khảo, báo cáo được tổ chức thành ba chương:

- **Chương 1. Phân tích và thiết kế cơ sở dữ liệu trên MongoDB:** phân tích bài toán, lựa chọn mô hình dữ liệu, thiết kế collection; tạo lập và thao tác dữ liệu.
- **Chương 2. Khai thác và tổng hợp dữ liệu trên MongoDB:** truy vấn theo nghiệp vụ, Aggregation Pipeline, \$lookup và đánh giá kết quả.
- **Chương 3. Triển khai cơ sở dữ liệu phân tán trong MongoDB:** Replica Set, lựa chọn khóa phân mảnh, Sharded Cluster và đánh giá kết quả.

CHƯƠNG 1. PHÂN TÍCH VÀ THIẾT KẾ CƠ SỞ DỮ LIỆU TRÊN MONGODB

Chương 1 trình bày quá trình phân tích bài toán quản lý thiết bị máy tính dựa trên ba bảng dữ liệu nguồn PRODUCT, LAPTOP và PRINTER. Từ kết quả phân tích, chương xác định các đối tượng, mối quan hệ và các mẫu truy cập dữ liệu chính, sau đó so sánh hai cách tiếp cận mô hình hóa của MongoDB là nhúng (Embedded) và tham chiếu (Reference) để lựa chọn mô hình phù hợp. Phần cuối chương trình bày thiết kế chi tiết các collection, quá trình tạo lập cơ sở dữ liệu, nhập dữ liệu nguồn và các thao tác tìm kiếm, cập nhật trên document, embedded document và mảng.

1.1. Phân tích bài toán và dữ liệu nguồn

1.1.1. Mô tả bài toán và dữ liệu cần quản lý

Bài toán đặt ra là xây dựng cơ sở dữ liệu phục vụ quản lý danh mục thiết bị máy tính của một đơn vị kinh doanh, phân phối thiết bị. Đơn vị này cung cấp ba nhóm sản phẩm: máy tính để bàn (PC), máy tính xách tay (laptop) và máy in (printer) đến từ tám nhà sản xuất (NSX) được ký hiệu từ A đến H. Mỗi sản phẩm được nhận diện bởi một mã model duy nhất, thuộc về đúng một NSX và một loại sản phẩm. Với laptop, đơn vị cần quản lý cấu hình (tốc độ bộ xử lý, dung lượng RAM, dung lượng ổ cứng, kích thước màn hình) và giá bán; với máy in, cần quản lý khả năng in màu, công nghệ in và giá bán.

Dữ liệu nguồn gồm ba bảng được trích từ lược đồ cơ sở dữ liệu sản phẩm trong [6] (Figures 2.20–2.21). Bảng **PRODUCT(maker, model, type)** lưu mã NSX (A – H), mã model duy nhất và loại sản phẩm (pc, laptop, printer). Bảng **LAPTOP(model, speed, ram, hd, screen, price)** lưu tốc độ bộ xử lý (GHz), dung lượng RAM (MB), ổ cứng (GB), kích thước màn hình (inch) và giá bán (USD). Bảng **PRINTER(model, color, type, price)** lưu khả năng in màu (true/false), công nghệ in (ink-jet – phun mực, laser) và giá bán (USD). Thuộc tính model của LAPTOP và PRINTER tham chiếu tới PRODUCT. Dữ liệu cụ thể được trình bày trong Bảng 1.1, Bảng 1.2 và Bảng 1.3.

Bảng 1.1. Dữ liệu bảng PRODUCT (30 bản ghi)

maker	model	type	maker	model	type	maker	model	type
A	1001	pc	C	1007	pc	E	2002	laptop
A	1002	pc	D	1008	pc	E	2003	laptop
A	1003	pc	D	1009	pc	E	3001	printer
A	2004	laptop	D	1010	pc	E	3002	printer

maker	model	type	maker	model	type	maker	model	type
A	2005	laptop	D	3004	printer	E	3003	printer
A	2006	laptop	D	3005	printer	F	2008	laptop
B	1004	pc	E	1011	pc	F	2009	laptop
B	1005	pc	E	1012	pc	G	2010	laptop
B	1006	pc	E	1013	pc	H	3006	printer
B	2007	laptop	E	2001	laptop	H	3007	printer

Bảng 1.2. Dữ liệu bảng LAPTOP (10 bản ghi)

model	speed	ram	hd	screen	price
2001	2.00	2048	240	20.1	3673
2002	1.73	1024	80	17.0	949
2003	1.80	512	60	15.4	549
2004	2.00	512	60	13.3	1150
2005	2.16	1024	120	17.0	2500
2006	2.00	2048	80	15.4	1700
2007	1.83	1024	120	13.3	1429
2008	1.60	1024	100	15.4	900
2009	1.60	512	80	14.1	680
2010	2.00	2048	160	15.4	2300

Bảng 1.3. Dữ liệu bảng PRINTER (7 bản ghi)

model	color	type	price
3001	true	ink-jet	99
3002	false	laser	239
3003	true	laser	899
3004	true	ink-jet	120
3005	false	laser	120
3006	true	ink-jet	100
3007	true	laser	200

Thống kê sơ bộ từ bảng PRODUCT (Bảng 1.4) cho thấy 30 sản phẩm gồm 13 PC, 10 laptop và 7 máy in. NSX E có nhiều sản phẩm nhất (9 sản phẩm) và là NSX duy nhất cung cấp đủ cả ba loại; NSX C và G mỗi NSX chỉ có một sản phẩm.

Bảng 1.4. Thống kê số sản phẩm theo nhà sản xuất và loại sản phẩm

NSX	pc	laptop	printer	Tổng
A	3	3	0	6
B	3	1	0	4

NSX	pc	laptop	printer	Tổng
C	1	0	0	1
D	3	0	2	5
E	3	3	3	9
F	0	2	0	2
G	0	1	0	1
H	0	0	2	2
Tổng	13	10	7	30

Qua khảo sát dữ liệu nguồn, có thể rút ra một số đặc điểm quan trọng ảnh hưởng tới thiết kế:

- Dữ liệu nguồn không có bảng chi tiết cho PC; 13 sản phẩm loại pc chỉ có ba thuộc tính maker, model, type. Theo yêu cầu đề tài, không tự ý bổ sung thuộc tính hay giá trị cho các sản phẩm này.
- Thuộc tính type xuất hiện với hai ý nghĩa khác nhau: PRODUCT.type là loại sản phẩm, còn PRINTER.type là công nghệ in. Khi hợp nhất dữ liệu cần tránh xung đột tên.
- Thuộc tính giá (price) chỉ có ở laptop và máy in; các thuộc tính cấu hình của laptop và máy in hoàn toàn khác nhau – dữ liệu có tính không đồng nhất (heterogeneous).
- Mọi model trong LAPTOP (2001–2010) và PRINTER (3001–3007) đều tồn tại trong PRODUCT với loại tương ứng; toàn vẹn tham chiếu giữa các bảng nguồn được thỏa mãn.
- Các giá trị speed như 2.00, 1.80 và screen như 17.0 là số thực; khi lưu bằng kiểu double, MongoDB hiển thị 2, 1.8, 17 nhưng giá trị không thay đổi.

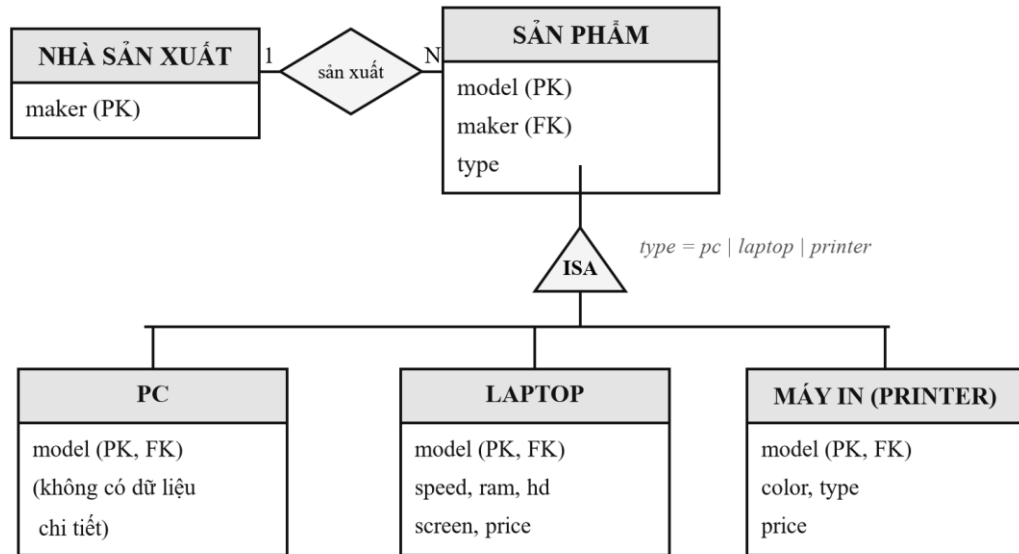
1.1.2. Xác định các đối tượng và mối quan hệ

Từ mô tả bài toán và dữ liệu nguồn, các đối tượng (thực thể) cần quản lý được xác định như sau:

- **Nhà sản xuất (NSX):** được nhận diện bởi mã maker; trong dữ liệu nguồn NSX không có thêm thuộc tính nào khác ngoài mã.
- **Sản phẩm:** thực thể tổng quát, gồm model (khóa), maker và type.
- **Laptop, Máy in, PC:** các thực thể chuyên biệt của Sản phẩm. Laptop có speed, ram, hd, screen, price; Máy in có color, type, price; PC không có dữ liệu chi tiết.

Giữa các đối tượng tồn tại hai loại mối quan hệ (Hình 1.1):

- **NSX – Sản phẩm (1 – N):** một NSX sản xuất nhiều sản phẩm, mỗi sản phẩm thuộc đúng một NSX. Theo dữ liệu, số sản phẩm của mỗi NSX dao động từ 1 (C, G) đến 9 (E).
- **Sản phẩm – Laptop / Máy in / PC (quan hệ tổng quát hóa – chuyên biệt hóa, ISA):** phân loại theo thuộc tính type; các lớp con rời nhau (một sản phẩm chỉ thuộc một loại) và toàn phần (mọi sản phẩm đều thuộc một loại). Giữa một sản phẩm và bản ghi chi tiết tương ứng là quan hệ 1 – 1.



Hình 1.1. Sơ đồ thực thể – mối quan hệ của bài toán quản lý thiết bị máy tính

Khác với thiết kế quan hệ (xuất phát từ cấu trúc dữ liệu), thiết kế lược đồ trong MongoDB xuất phát từ cách ứng dụng truy cập dữ liệu [3], [4]. Vì vậy, bên cạnh các đối tượng và mối quan hệ, cần xác định các mẫu truy cập (access pattern) chính của nghiệp vụ như trong Bảng 1.5.

Bảng 1.5. Các mẫu truy cập dữ liệu chính của nghiệp vụ

Mã	Mẫu truy cập	Tần suất
TC1	Tra cứu một sản phẩm theo model, kèm đầy đủ cấu hình và giá	Rất thường xuyên
TC2	Liệt kê các sản phẩm của một NSX	Thường xuyên
TC3	Lọc sản phẩm theo loại và điều kiện cấu hình, giá	Thường xuyên
TC4	Thống kê số lượng, giá theo NSX và loại sản phẩm	Định kỳ
TC5	Xem danh mục (portfolio) sản phẩm của từng NSX	Thường xuyên
TC6	Cập nhật giá, cấu hình của một sản phẩm	Thỉnh thoảng
TC7	Bổ sung sản phẩm mới cho một NSX	Thỉnh thoảng

Đồng thời, các ràng buộc nghiệp vụ cần được bảo đảm gồm:

- Mỗi sản phẩm bắt buộc có maker, model, type; model là duy nhất trên toàn bộ danh mục.
- type chỉ nhận một trong ba giá trị pc, laptop, printer; giá bán (nếu có) không âm.
- Thông tin cấu hình phải phù hợp với loại sản phẩm: laptop có speed, ram, hd, screen; máy in có color và công nghệ in.

1.2. Phân tích và lựa chọn mô hình dữ liệu MongoDB

MongoDB lưu trữ dữ liệu dưới dạng các document BSON trong các collection. Lược đồ của collection linh hoạt: các document trong cùng collection có thể có tập trường khác nhau, và một trường có thể chứa document con (embedded document) hoặc mảng (array). Để biểu diễn mối quan hệ giữa các đối tượng, MongoDB cung cấp hai cách tiếp cận cơ bản là **nhúng dữ liệu** (embedding) và **tham chiếu dữ liệu** (referencing) [4], [7]. Nguyên tắc chung là: dữ liệu thường được truy cập cùng nhau thì nên được lưu cùng nhau.

1.2.1. Phân tích khả năng sử dụng *Embedded Model*

Trong mô hình nhúng, dữ liệu liên quan được lưu trực tiếp bên trong một document dưới dạng embedded document hoặc mảng các embedded document. Với bài toán này có hai phương án nhúng khả dĩ (Hình 1.2):

- **Phương án N1 – nhúng theo sản phẩm:** mỗi sản phẩm là một document trong collection products; thông tin chi tiết của LAPTOP hoặc PRINTER được nhúng vào trường specs, giá được đưa lên cấp gốc.
- **Phương án N2 – nhúng theo NSX:** mỗi NSX là một document trong collection makers, chứa mảng products gồm toàn bộ sản phẩm của NSX đó cùng cấu hình và giá.

Phương án N1: nhúng theo sản phẩm	Phương án N2: nhúng theo nhà SX
<pre>// collection products { maker: "E", model: 2001, type: "laptop", specs: { speed: 2.00, ram: 2048, hd: 240, screen: 20.1 }, price: 3673 } { maker: "E", model: 3001, type: "printer", specs: { color: true, type: "ink-jet" }, price: 99 }</pre>	<pre>// collection makers { _id: "E", products: [{ model: 1011, type: "pc" }, { model: 2001, type: "laptop", specs: {...}, price: 3673 }, { model: 3001, type: "printer", specs: {...}, price: 99 }, ... (9 phần tử)] }</pre>

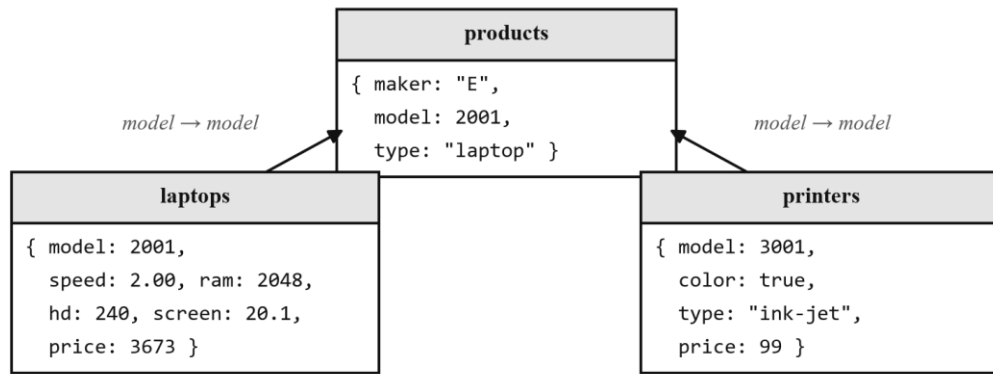
Hình 1.2. Hai phương án mô hình nhúng (Embedded Model)

Đánh giá phương án N1. Quan hệ giữa sản phẩm và bản ghi chi tiết là 1 – 1, dữ liệu chi tiết luôn được đọc cùng sản phẩm (TC1, TC3) nên việc nhúng là tự nhiên: một lần đọc lấy đủ thông tin, không cần phép nối; thao tác ghi trên một sản phẩm là nguyên tử vì chỉ tác động lên một document. Phương án này áp dụng mẫu thiết kế đa hình (polymorphic pattern): các loại sản phẩm khác nhau cùng nằm trong một collection, mỗi loại có cấu trúc specs riêng; PC không có specs vẫn là document hợp lệ. Kích thước document nhỏ và ổn định.

Đánh giá phương án N2. Phương án này thuận lợi cho TC5 (xem danh mục của NSX) nhưng bộc lộ nhiều hạn chế: tra cứu một sản phẩm phải tìm trong mảng và dùng toán tử chiếu hoặc \$unwind; cập nhật một phần tử mảng cần toán tử vị trí; mảng tăng không giới hạn khi NSX bổ sung sản phẩm (mẫu phản thiết kế “unbounded array”); toàn bộ dữ liệu chỉ nằm trong 8 document với kích thước chênh lệch lớn (E có 9 sản phẩm), gây khó khăn khi phân mảnh vì đơn vị phân bố nhỏ nhất là một document NSX.

1.2.2. Phân tích khả năng sử dụng Reference/Normalized Model

Trong mô hình tham chiếu (chuẩn hóa), các đối tượng được lưu ở các collection riêng và liên kết với nhau thông qua giá trị khóa, tương tự khóa ngoại trong mô hình quan hệ. Áp dụng cho bài toán, phương án R ánh xạ trực tiếp ba bảng nguồn thành ba collection products, laptops, printers, liên kết qua trường model (Hình 1.3).



Hình 1.3. Phương án mô hình tham chiếu (Reference Model)

Ưu điểm của phương án R là bám sát cấu trúc dữ liệu nguồn, không trùng lặp dữ liệu, mỗi collection nhỏ và có thể cập nhật độc lập. **Nhược điểm** là các mẫu truy cập chính TC1, TC3 đều cần kết hợp products với laptops hoặc printers bằng \$lookup, làm tăng chi phí xử lý; MongoDB không kiểm tra ràng buộc khóa ngoại nên toàn vẹn tham chiếu phụ thuộc vào ứng dụng; việc thêm một sản phẩm cần ghi vào hai collection nên muốn bảo đảm nguyên tử phải dùng giao tác đa tài liệu; ngoài ra, khi các collection được phân mảnh, phép nối giữa các shard tốn thêm chi phí truyền dữ liệu qua mạng.

Mô hình tham chiếu phù hợp với quan hệ 1 – N có N lớn hoặc không giới hạn, quan hệ N – N, hoặc khi các đối tượng thường được truy cập độc lập. Trong bài toán, quan hệ NSX – Sản phẩm mang đặc điểm này hơn là quan hệ Sản phẩm – Laptop/Máy in.

1.2.3. Lựa chọn và giải thích mô hình dữ liệu

Bảng 1.6 tổng hợp kết quả so sánh ba phương án theo các mẫu truy cập và tiêu chí quản trị đã xác định.

Bảng 1.6. So sánh các phương án mô hình dữ liệu

Tiêu chí	N1 – nhúng theo sản phẩm	N2 – nhúng theo NSX	R – tham chiếu
Đọc một sản phẩm kèm cấu hình (TC1)	1 document, không nối	Lọc trong mảng	Cần \$lookup
Lọc theo cấu hình, giá (TC3)	Truy vấn trực tiếp, đánh chỉ mục được	Cần \$unwind	Cần \$lookup
Thống kê theo NSX, loại (TC4)	\$group trực tiếp	\$unwind rồi \$group	\$group (+ \$lookup nếu cần giá)
Xem danh mục của NSX (TC5)	Lọc theo maker	1 document	Lọc theo maker
Tính nguyên tử khi ghi	1 document	1 document lớn dần	Nhiều document

Tiêu chí	N1 – nhúng theo sản phẩm	N2 – nhúng theo NSX	R – tham chiếu
Kích thước document	Nhỏ, ổn định	Tăng theo số sản phẩm	Nhỏ
Khả năng phân mảnh	Tốt (maker, model ở cấp gốc)	Kém (chỉ 8 document)	Trung bình (nối liền shard)

Trên cơ sở phân tích, tiểu luận lựa chọn **mô hình kết hợp (hybrid)** giữa nhúng và tham chiếu:

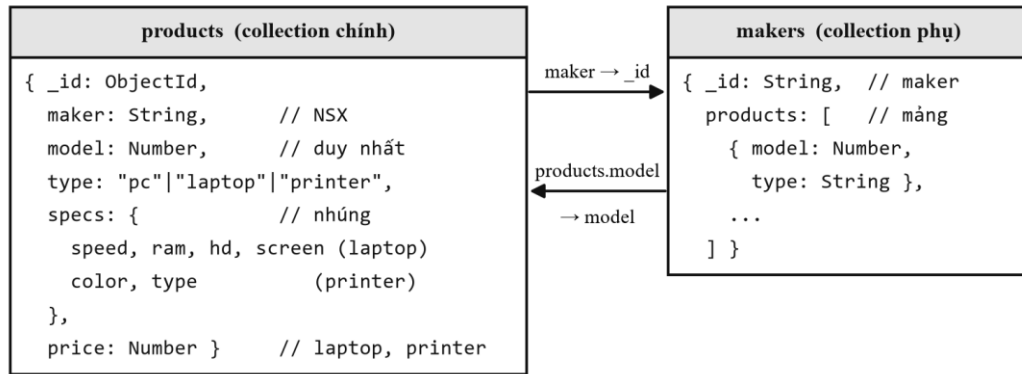
- **Collection chính `products` theo phương án N1:** thông tin chi tiết LAPTOP/PRINTER được nhúng vào embedded document specs (quan hệ 1 – 1, luôn đọc cùng nhau); giá price đặt ở cấp gốc để truy vấn, thống kê giá thống nhất cho mọi loại. Tên thuộc tính gốc được giữ nguyên, kể cả PRINTER.type được giữ thành specs.type – không xung đột với type ở cấp gốc vì nằm ở cấp lồng khác nhau.
- **Collection phụ `makers` theo hướng tham chiếu:** mỗi NSX là một document có mảng products gồm các phần tử { model, type }; trong đó model là tham chiếu tới products.model và type được lưu kèm theo mẫu tham chiếu mở rộng (extended reference) để trả lời nhanh câu hỏi “NSX có những loại sản phẩm nào”. Mảng có kích thước nhỏ (tối đa 9 phần tử), phục vụ TC5 và các truy vấn kết hợp bằng \$lookup.
- Collection makers không chứa dữ liệu mới: nó được sinh hoàn toàn từ dữ liệu PRODUCT bằng Aggregation Pipeline, bảo đảm khớp với dữ liệu nguồn.

Sự đánh đổi của lựa chọn này là quan hệ NSX – Sản phẩm được lưu ở hai nơi (trường maker của products và mảng products của makers), do đó khi bổ sung sản phẩm mới cần cập nhật cả hai collection trong cùng một giao tác hoặc sinh lại makers bằng \$merge. Với đặc thù danh mục sản phẩm có tần suất đọc cao và ít thay đổi (TC6, TC7 chỉ “thỉnh thoảng”), sự đánh đổi này là hợp lý.

1.3. Thiết kế cấu trúc cơ sở dữ liệu MongoDB

1.3.1. Xác định các collection và mối liên kết

Cơ sở dữ liệu được đặt tên QLThietBi, gồm hai collection (Hình 1.4): products (30 document) lưu sản phẩm cùng cấu hình nhúng và giá, nhận diện bởi model (chỉ mục duy nhất), liên kết tới NSX qua maker → makers._id; makers (8 document) lưu danh mục sản phẩm theo NSX với _id là mã NSX, liên kết tới sản phẩm qua products.model → products.model.



specs: embedded document (quan hệ 1-1) products[]: mảng tham chiếu (quan hệ 1-N)

Hình 1.4. Mô hình dữ liệu MongoDB được lựa chọn

Việc ánh xạ từ các bảng nguồn sang cấu trúc MongoDB được trình bày trong Bảng 1.7. Có thể thấy toàn bộ thuộc tính nguồn đều có vị trí tương ứng, không thuộc tính nào bị loại bỏ hay đổi tên.

Bảng 1.7. Ánh xạ thuộc tính nguồn sang cấu trúc MongoDB

Thuộc tính nguồn	Vị trí trong MongoDB
PRODUCT.maker	products.maker; makers._id
PRODUCT.model	products.model; makers.products[].model
PRODUCT.type	products.type; makers.products[].type
LAPTOP.model, PRINTER.model	Hợp nhất với products.model (quan hệ 1 – 1)
LAPTOP.speed, ram, hd, screen	products.specs.speed, .ram, .hd, .screen
PRINTER.color, PRINTER.type	products.specs.color, products.specs.type
LAPTOP.price, PRINTER.price	products.price

1.3.2. Thiết kế document, embedded document, array và reference

Cấu trúc document của collection products được mô tả trong Bảng 1.8. Đây là document đa hình: trường specs là embedded document có cấu trúc phụ thuộc vào type.

Bảng 1.8. Cấu trúc document của collection products

Trường	Kiểu BSON	Bắt buộc	Mô tả
_id	ObjectId	Tự sinh	Khóa nội bộ do MongoDB sinh tự động
maker	string	Có	Mã NSX; tham chiếu makers._id
model	number	Có	Mã model; chỉ mục duy nhất uq_model
type	string	Có	Loại sản phẩm: "pc" "laptop" "printer"
specs	object	Không	Embedded document. Laptop: { speed, ram, hd, screen }; máy in: { color, type }; PC: không có
price	number	Không	Giá bán (USD), ≥ 0 ; chỉ có ở laptop và máy in

Document của collection makers gồm _id (string – mã NSX) và products (array) – mảng các embedded document { model: number, type: string }, trong đó model tham chiếu tới products.model.

model được giữ là trường nghiệp vụ riêng (không dùng làm _id) để bám sát dữ liệu nguồn, tính duy nhất được bảo đảm bằng chỉ mục uq_model. Ràng buộc nghiệp vụ được khai báo bằng \$jsonSchema ở mức collection (validationLevel: "strict", validationAction: "error"). Giá trị số được lưu bằng kiểu double mặc định của mongosh – biểu diễn chính xác với dữ liệu giá là số nguyên nhỏ; hệ thống tài chính thực tế nên cân nhắc kiểu Decimal128.

1.3.3. Minh họa các document mẫu

Các document đại diện cho ba loại sản phẩm trong products và một document của makers (dạng đã nhập vào cơ sở dữ liệu):

```
// products - PC (không có specs, không có price)
{ maker: "A", model: 1001, type: "pc" }
// products - laptop (specs là embedded document)
{ maker: "E", model: 2001, type: "laptop",
  specs: { speed: 2.00, ram: 2048, hd: 240, screen: 20.1 },
  price: 3673 }
// products - máy in (specs.type là công nghệ in)
{ maker: "E", model: 3003, type: "printer",
  specs: { color: true, type: "laser" },
  price: 899 }
// makers - NSX E (products là mảng các tham chiếu)
{ _id: "E",
  products: [ { model: 1011, type: "pc" }, { model: 1012, type: "pc" },
               { model: 1013, type: "pc" }, { model: 2001, type: "laptop" },
               { model: 2002, type: "laptop" }, { model: 2003, type: "laptop" },
               { model: 3001, type: "printer" }, { model: 3002, type: "printer" },
               { model: 3003, type: "printer" } ] }
```

1.4. Tạo lập và thao tác dữ liệu

Môi trường thực nghiệm được mô tả trong Bảng 1.9. Toàn bộ lệnh trong mục này được thực hiện bằng mongosh kết nối tới nút Primary mongo1 của Replica Set rs0 (cấu hình chi tiết của Replica Set trình bày ở mục 3.1). Kết quả trình bày trong báo

cáo là kết quả thực thi thực tế; các khối lệnh có nền xám, các khối kết quả có viền nét đứt.

Bảng 1.9. Môi trường thực nghiệm

Thành phần	Phiên bản / cấu hình
Hệ quản trị CSDL	MongoDB Community Server 8.0.32 (Docker image mongo:8.0)
Công cụ dòng lệnh	mongosh 2.12.0
Nền tảng triển khai	Docker Engine 28.5.1, Docker Compose; Windows 11 Pro
Cụm sao chép	Replica Set rs0: mongo1, mongo2, mongo3 (cổng 27017)
Cụm phân mảnh	cfgrs (3 nút) + shard1rs (3 nút) + shard2rs (3 nút) + mongos
Cơ sở dữ liệu	QLThietBi

1.4.1. Tạo collection và nhập dữ liệu

Bước 1: Tạo collection products với bộ kiểm tra dữ liệu

Chọn cơ sở dữ liệu QLThietBi (MongoDB tạo cơ sở dữ liệu khi có collection đầu tiên) và tạo collection products kèm bộ kiểm tra \$jsonSchema:

```
use QLThietBi
db.createCollection("products", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["maker", "model", "type"],
      properties: {
        maker: { bsonType: "string" },
        model: { bsonType: "number" },
        type: { enum: ["pc", "laptop", "printer"] },
        price: { bsonType: "number", minimum: 0 },
        specs: { bsonType: "object" }
      }
    }
  }
})
```

Kết quả:

```
{ ok: 1 }
```

Bước 2: Tạo chỉ mục duy nhất trên model

```
db.products.createIndex({ model: 1 }, { unique: true, name: "uq_model" })
```

```
uq_model
```

Bước 3: Nhập dữ liệu nguồn

Dữ liệu ba bảng nguồn được hợp nhất và nhập vào products bằng một lệnh insertMany gồm 30 document, đúng thứ tự và giá trị của bảng PRODUCT; với laptop và máy in, các thuộc tính chi tiết được nhúng vào specs:

```
db.products.insertMany([
  { maker: "A", model: 1001, type: "pc" },
```

```

{ maker: "A", model: 1002, type: "pc" },
{ maker: "A", model: 1003, type: "pc" },
{ maker: "A", model: 2004, type: "laptop",
  specs: { speed: 2.00, ram: 512, hd: 60, screen: 13.3 }, price: 1150 },
{ maker: "A", model: 2005, type: "laptop",
  specs: { speed: 2.16, ram: 1024, hd: 120, screen: 17.0 }, price: 2500 },
{ maker: "A", model: 2006, type: "laptop",
  specs: { speed: 2.00, ram: 2048, hd: 80, screen: 15.4 }, price: 1700 },
{ maker: "B", model: 1004, type: "pc" },
{ maker: "B", model: 1005, type: "pc" },
{ maker: "B", model: 1006, type: "pc" },
{ maker: "B", model: 2007, type: "laptop",
  specs: { speed: 1.83, ram: 1024, hd: 120, screen: 13.3 }, price: 1429 },
{ maker: "C", model: 1007, type: "pc" },
{ maker: "D", model: 1008, type: "pc" },
{ maker: "D", model: 1009, type: "pc" },
{ maker: "D", model: 1010, type: "pc" },
{ maker: "D", model: 3004, type: "printer",
  specs: { color: true, type: "ink-jet" }, price: 120 },
{ maker: "D", model: 3005, type: "printer",
  specs: { color: false, type: "laser" }, price: 120 },
{ maker: "E", model: 1011, type: "pc" },
{ maker: "E", model: 1012, type: "pc" },
{ maker: "E", model: 1013, type: "pc" },
{ maker: "E", model: 2001, type: "laptop",
  specs: { speed: 2.00, ram: 2048, hd: 240, screen: 20.1 }, price: 3673 },
{ maker: "E", model: 2002, type: "laptop",
  specs: { speed: 1.73, ram: 1024, hd: 80, screen: 17.0 }, price: 949 },
{ maker: "E", model: 2003, type: "laptop",
  specs: { speed: 1.80, ram: 512, hd: 60, screen: 15.4 }, price: 549 },
{ maker: "E", model: 3001, type: "printer",
  specs: { color: true, type: "ink-jet" }, price: 99 },
{ maker: "E", model: 3002, type: "printer",
  specs: { color: false, type: "laser" }, price: 239 },
{ maker: "E", model: 3003, type: "printer",
  specs: { color: true, type: "laser" }, price: 899 },
{ maker: "F", model: 2008, type: "laptop",
  specs: { speed: 1.60, ram: 1024, hd: 100, screen: 15.4 }, price: 900 },
{ maker: "F", model: 2009, type: "laptop",
  specs: { speed: 1.60, ram: 512, hd: 80, screen: 14.1 }, price: 680 },
{ maker: "G", model: 2010, type: "laptop",
  specs: { speed: 2.00, ram: 2048, hd: 160, screen: 15.4 }, price: 2300 },
{ maker: "H", model: 3006, type: "printer",
  specs: { color: true, type: "ink-jet" }, price: 100 },
{ maker: "H", model: 3007, type: "printer",
  specs: { color: true, type: "laser" }, price: 200 }
]

```

1)

MongoDB xác nhận ghi thành công (acknowledged: true, sinh 30 ObjectId).

Kiểm tra số lượng document tổng và theo loại:

```

{
  acknowledged: true,
  insertedIds: {
    ...
  }
}

```

```
db.products.countDocuments()
```

30

```

db.products.aggregate([
  { $group: { _id: "$type", soLuong: { $sum: 1 } } },
  { $sort: { _id: 1 } }
])

```

```
[
  { _id: 'laptop', soLuong: 10 },
  { _id: 'pc', soLuong: 13 },
  { _id: 'printer', soLuong: 7 }
]
```

Có 30 document gồm 13 PC, 10 laptop và 7 máy in – khớp hoàn toàn với Bảng 1.4.

Bước 4: Tạo collection makers từ dữ liệu products

Collection makers được sinh bằng Aggregation Pipeline: sắp xếp theo model, nhóm theo maker, đẩy từng cặp { model, type } vào mảng và ghi kết quả ra collection bằng \$out:

```
db.products.aggregate([
  { $sort: { model: 1 } },
  { $group: { _id: "$maker", products: { $push: { model: "$model",
    type: "$type" } } } },
  { $sort: { _id: 1 } },
  { $out: "makers" }
])
```

Kiểm tra kết quả – số phần tử mảng products của từng NSX:

```
db.makers.find({}, { _id: 1, soSanPham: { $size: "$products" } })
```

```
[
  { _id: 'A', soSanPham: 6 },
  { _id: 'B', soSanPham: 4 },
  { _id: 'C', soSanPham: 1 },
  { _id: 'D', soSanPham: 5 },
  { _id: 'E', soSanPham: 9 },
  { _id: 'F', soSanPham: 2 },
  { _id: 'G', soSanPham: 1 },
  { _id: 'H', soSanPham: 2 }
]
```

Số phần tử mảng của từng NSX (6, 4, 1, 5, 9, 2, 1, 2) khớp với cột “Tổng” của Bảng 1.4, chứng tỏ makers phản ánh chính xác dữ liệu PRODUCT.

Bước 5: Kiểm tra các ràng buộc toàn vẹn

Hai thao tác ghi vi phạm ràng buộc được thử nghiệm (Hình 1.5): (1) thêm sản phẩm có type: "tablet" không thuộc tập giá trị cho phép – bị bộ kiểm tra \$jsonSchema từ chối với thông báo “Document failed validation” và chỉ rõ thuộc tính vi phạm; (2) thêm sản phẩm trùng model 1001 – bị chỉ mục duy nhất uq_model từ chối với lỗi E11000. Cả hai thao tác đều không làm thay đổi dữ liệu.

```

○ ○ ○ mongosh — mongo1:27017 (Replica Set rs0)

rs0 [direct: primary] QLThietBi> db.products.insertOne({ maker: "Z", model: 9001, type: "tablet" })
Uncaught:

MongoServerError: Document failed validation
...
  {
    propertyName: 'type',
    description: 'Loại sản phẩm',
    details: [
      {
        operatorName: 'enum',
        specifiedAs: { enum: [ 'pc', 'laptop', 'printer' ] },
        reason: 'value was not found in enum',
        consideredValue: 'tablet'
      }
    ]
  }

rs0 [direct: primary] QLThietBi> db.products.insertOne({ maker: "A", model: 1001, type: "pc" })
Uncaught
MongoServerError: E11000 duplicate key error collection: QLThietBi.products index: uq_model dup key: {
  model: 1001 }

```

Hình 1.5. Kết quả kiểm tra ràng buộc \$jsonSchema và chỉ mục duy nhất

1.4.2. Thực hiện tìm kiếm và cập nhật dữ liệu

a) Tìm kiếm dữ liệu

Tra cứu một sản phẩm theo model (TC1) bằng `findOne` – toàn bộ thông tin cấu hình và giá được trả về trong một document:

```

db.products.findOne({ model: 2001 })

{
  _id: ObjectId('6ac352db92db87c20489e4f3'),
  maker: 'E',
  model: 2001,
  type: 'laptop',
  specs: { speed: 2, ram: 2048, hd: 240, screen: 20.1 },
  price: 3673
}

```

Liệt kê sản phẩm của NSX B (TC2) với phép chiếu chỉ lấy các trường cần thiết:

```

db.products.find({ maker: "B" }, { _id: 0, maker: 1, model: 1, type: 1, price: 1 })

[
  { maker: 'B', model: 1004, type: 'pc' },
  { maker: 'B', model: 1005, type: 'pc' },
  { maker: 'B', model: 1006, type: 'pc' },
  { maker: 'B', model: 2007, type: 'laptop', price: 1429 }
]

```

Ba PC của B không có trường `price` trong kết quả vì dữ liệu nguồn không có giá cho PC – mô hình document cho phép bỏ qua trường không có giá trị thay vì lưu giá trị rỗng (`null`).

b) Cập nhật dữ liệu

Đề tài yêu cầu không được thay đổi dữ liệu nguồn, vì vậy mỗi thao tác cập nhật dưới đây đều được thực hiện để minh họa, kiểm tra kết quả, sau đó **hoàn tác về giá trị gốc**. Cập nhật giá laptop 2002 bằng `updateOne` với toán tử `$set`:

```
db.products.updateOne({ model: 2002 }, { $set: { price: 999 } })
```

```
{
  ...
  matchedCount: 1,
  modifiedCount: 1,
  ...
}
```

```
db.products.findOne({ model: 2002 }, { _id: 0, model: 1, price: 1 })
```

```
{ model: 2002, price: 999 }
```

Hoàn tác về giá gốc 949 bằng lệnh tương tự với { \$set: { price: 949 } }. Tiếp theo, cập nhật nhiều document: tăng giá 10 USD cho tất cả máy in laser bằng updateMany với toán tử \$inc (điều kiện lọc dùng ký hiệu dấu chấm trên embedded document):

```
db.products.updateMany(
  { type: "printer", "specs.type": "laser" },
  { $inc: { price: 10 } }
)
```

```
{
  ...
  matchedCount: 4,
  modifiedCount: 4,
  ...
}
```

```
db.products.find(
  { type: "printer", "specs.type": "laser" },
  { _id: 0, model: 1, price: 1 }
).sort({ model: 1 })
```

```
[
  { model: 3002, price: 249 },
  { model: 3003, price: 909 },
  { model: 3005, price: 130 },
  { model: 3007, price: 210 }
]
```

Bốn máy in laser (3002, 3003, 3005, 3007) đều được tăng giá; thao tác được hoàn tác bằng updateMany cùng điều kiện với { \$inc: { price: -10 } }.

Bộ kiểm tra \$jsonSchema cũng áp dụng cho thao tác cập nhật: lệnh updateOne({ model: 3001 }, { \$set: { price: -5 } }) bị từ chối với lỗi “Document failed validation” do vi phạm điều kiện minimum: 0.

Cuối cùng, sau toàn bộ các thao tác minh họa ở mục 1.4.2 và 1.4.3, tổng số lượng và tổng giá theo loại được kiểm tra lại. Tổng giá laptop 15830 và tổng giá máy in 1777 trùng khớp với tổng cột price của Bảng 1.2 và Bảng 1.3, khẳng định dữ liệu đã được khôi phục chính xác:

```
db.products.aggregate([
  { $match: { price: { $exists: true } } },
  { $group: { _id: "$type", soLuong: { $sum: 1 }, tongGia: { $sum: "$price" } } },
  { $sort: { _id: 1 } }
])
```

```
[
  { _id: 'laptop', soLuong: 10, tongGia: 15830 },
  { _id: 'printer', soLuong: 7, tongGia: 1777 }
]
```

1.4.3. Thao tác với embedded document và array

a) Truy vấn và cập nhật embedded document

Truy vấn trên trường của embedded document dùng ký hiệu dấu chấm (dot notation). Tìm laptop có RAM từ 2048 MB:

```
db.products.find(
  { type: "laptop", "specs.ram": { $gte: 2048 } },
  { _id: 0, maker: 1, model: 1, "specs.ram": 1, "specs.hd": 1, price: 1 }
)
```

```
[
  { maker: 'A', model: 2006, specs: { ram: 2048, hd: 80 }, price: 1700 },
  { maker: 'E', model: 2001, specs: { ram: 2048, hd: 240 }, price: 3673 },
  { maker: 'G', model: 2010, specs: { ram: 2048, hd: 160 }, price: 2300 }
]
```

Tìm máy in màu sử dụng công nghệ laser – kết hợp hai điều kiện trên embedded document; kết quả gồm hai máy in 3003 (E, 899 USD) và 3007 (H, 200 USD):

```
db.products.find(
  { "specs.color": true, "specs.type": "laser" },
  { _id: 0, maker: 1, model: 1, specs: 1, price: 1 }
)
```

Toán tử \$exists cho phép lọc theo sự tồn tại của embedded document: countDocuments({ specs: { \$exists: false } }) trả về 13 – đúng bằng số PC.

Cập nhật một trường bên trong embedded document chỉ tác động tới trường đó, các trường khác của specs được giữ nguyên. Thao tác được hoàn tác ngay sau đó và kiểm tra lại:

```
db.products.updateOne({ model: 2003 }, { $set: { "specs.screen": 15.6 } })
// sau khi hoàn tác bằng { $set: { "specs.screen": 15.4 } }
db.products.findOne({ model: 2003 }, { _id: 0, model: 1, specs: 1, price: 1 })
```

```
{
  model: 2003,
  specs: { speed: 1.8, ram: 512, hd: 60, screen: 15.4 },
  price: 549
}
```

b) Truy vấn và cập nhật mảng

Truy vấn trên mảng embedded document của makers: điều kiện "products.type": "printer" đúng khi ít nhất một phần tử mảng thỏa mãn – kết quả là các NSX có sản xuất máy in:

```
db.makers.find({ "products.type": "printer" }, { _id: 1 })
```

```
[ { _id: 'D' }, { _id: 'E' }, { _id: 'H' } ]
```

Hình 1.6 minh họa ba toán tử mảng: \$elemMatch yêu cầu **cùng một phần tử** thỏa mãn đồng thời nhiều điều kiện (laptop có model từ 2008 – kết quả F, G); \$size lọc theo số phần tử mảng (NSX chỉ có một sản phẩm – C, G); \$all yêu cầu mảng chứa đủ các giá trị (NSX có cả ba loại sản phẩm – chỉ có E).

```

○○○ mongosh — mongo1:27017 (Replica Set rs0)

rs0 [direct: primary] QLThietBi> db.makers.find(
  { products: { $elemMatch: { type: "laptop", model: { $gte: 2008 } } } },
  { _id: 1, products: 1 }
)
[
  {
    _id: 'F',
    products: [ { model: 2008, type: 'laptop' }, { model: 2009, type: 'laptop' } ]
  },
  { _id: 'G', products: [ { model: 2010, type: 'laptop' } ] }
]

rs0 [direct: primary] QLThietBi> db.makers.find({ products: { $size: 1 } })
[
  { _id: 'C', products: [ { model: 1007, type: 'pc' } ] },
  { _id: 'G', products: [ { model: 2010, type: 'laptop' } ] }
]

rs0 [direct: primary] QLThietBi> db.makers.find({ "products.type": { $all: ["pc", "laptop", "printer"]
  } }, { _id: 1 })
[ { _id: 'E' } ]

```

Hình 1.6. Kết quả truy vấn mảng với \$elemMatch, \$size và \$all

Toán tử \$addToSet chỉ thêm phần tử nếu phần tử đó chưa tồn tại. Thêm lại phần tử { model: 1007, type: "pc" } vào NSX C không làm thay đổi dữ liệu (modifiedCount: 0):

```

db.makers.updateOne(
  { _id: "C" },
  { $addToSet: { products: { model: 1007, type: "pc" } } }
)

```

```

{
  ...
  matchedCount: 1,
  modifiedCount: 0,
  ...
}

```

Toán tử \$push thêm phần tử vào cuối mảng và \$pull loại bỏ phần tử theo điều kiện. Thêm một phần tử thử nghiệm vào NSX G rồi loại bỏ:

```

db.makers.updateOne(
  { _id: "G" },
  { $push: { products: { model: 9999, type: "laptop" } } }
)

```

```

// db.makers.findOne({ _id: "G" }) sau $push
{
  _id: 'G',
  products: [ { model: 2010, type: 'laptop' }, { model: 9999, type: 'laptop' } ]
}

```

```
db.makers.updateOne(
  { _id: "G" },
  { $pull: { products: { model: 9999 } } }
)
```

```
// db.makers.findOne({ _id: "G" }) sau $pull
{ _id: 'G', products: [ { model: 2010, type: 'laptop' } ] }
```

Kết luận Chương 1

Chương 1 đã phân tích bài toán quản lý thiết bị máy tính và dữ liệu nguồn gồm 30 sản phẩm, 10 laptop và 7 máy in; xác định các đối tượng NSX, Sản phẩm, Laptop, Máy in, PC cùng quan hệ 1 – N giữa NSX và Sản phẩm, quan hệ chuyên biệt hóa (1 – 1) giữa Sản phẩm và các loại cụ thể. Trên cơ sở so sánh mô hình nhúng và mô hình tham chiếu theo các mẫu truy cập, tiểu luận lựa chọn mô hình kết hợp: collection products nhúng thông tin cấu hình vào specs, collection makers lưu mảng tham chiếu tới sản phẩm. Cơ sở dữ liệu QLThietBi đã được tạo lập với ràng buộc \$jsonSchema và chỉ mục duy nhất; dữ liệu nguồn được nhập đầy đủ, chính xác và được kiểm chứng bằng thống kê. Các thao tác tìm kiếm, cập nhật trên document, embedded document và mảng đều thực hiện thành công, dữ liệu được khôi phục nguyên trạng sau các thao tác minh họa – là nền tảng cho việc khai thác dữ liệu ở Chương 2.

CHƯƠNG 2. KHAI THÁC VÀ TỔNG HỢP DỮ LIỆU TRÊN MONGODB

Trên cơ sở dữ liệu đã được thiết kế và tạo lập ở Chương 1, Chương 2 tập trung vào việc khai thác dữ liệu phục vụ nghiệp vụ quản lý thiết bị máy tính. Trước hết, chương xác định danh sách 16 yêu cầu khai thác điển hình và hiện thực các yêu cầu tra cứu bằng phương thức `find`. Tiếp theo, chương trình bày việc tổng hợp dữ liệu bằng Aggregation Pipeline với các stage lọc, chiếu, nhóm, sắp xếp và phân khúc. Cuối cùng, chương sử dụng `$lookup` để kết hợp hai collection makers và products, đồng thời phân tích, đánh giá kết quả khai thác so với dữ liệu nguồn.

2.1. Xây dựng và thực hiện các yêu cầu truy vấn

2.1.1. Xác định các yêu cầu khai thác theo nghiệp vụ

Từ các mẫu truy cập TC1 – TC7 đã xác định ở Bảng 1.5, 16 yêu cầu khai thác dữ liệu (YC) được xây dựng, bao quát các nhu cầu tư vấn bán hàng, báo cáo thống kê và phân tích danh mục sản phẩm (Bảng 2.1).

Bảng 2.1. Danh sách yêu cầu khai thác dữ liệu theo nghiệp vụ

Mã	Yêu cầu nghiệp vụ	Kỹ thuật MongoDB	Mục
YC1	Liệt kê laptop giá dưới 1.000 USD, sắp xếp theo giá tăng dần	<code>find</code> , <code>\$lt</code> , <code>sort</code>	2.1.2
YC2	Tìm các NSX có laptop với ổ cứng từ 100 GB trở lên	<code>distinct</code> , <code>dot notation</code>	2.1.2
YC3	Tìm laptop màn hình từ 15,4 inch và RAM từ 1024 MB	<code>find</code> , điều kiện kết hợp	2.1.2
YC4	Liệt kê sản phẩm của NSX E có giá từ 200 đến 1.000 USD	<code>find</code> , <code>\$gte</code> , <code>\$lte</code> , <code>countDocuments</code>	2.1.2
YC5	Báo cáo cấu hình laptop: quy đổi RAM sang GB, phân loại hiệu năng	<code>\$match</code> , <code>\$project</code> , <code>\$divide</code> , <code>\$cond</code>	2.2.1
YC6	Danh sách máy in kèm công nghệ và nhãn chế độ in	<code>\$match</code> , <code>\$project</code> , <code>\$cond</code>	2.2.1
YC7	Thống kê số sản phẩm theo NSX và loại	<code>\$group</code> (khóa ghép), <code>\$sum</code>	2.2.2
YC8	Thống kê giá laptop theo NSX: trung bình, thấp nhất, cao nhất	<code>\$group</code> , <code>\$avg</code> , <code>\$min</code> , <code>\$max</code>	2.2.2
YC9	Thống kê theo loại sản phẩm: số lượng, tổng giá, giá trung bình	<code>\$group</code> , <code>\$cond</code> , <code>\$ifNull</code>	2.2.2
YC10	Tìm dung lượng ổ cứng xuất hiện ở từ hai laptop trở lên	<code>\$group</code> , <code>\$match</code> sau nhóm	2.2.2
YC11	Tìm 3 sản phẩm có giá cao nhất	<code>\$sort</code> , <code>\$limit</code>	2.2.3

Mã	Yêu cầu nghiệp vụ	Kỹ thuật MongoDB	Mục
YC12	Phân khúc laptop theo khoảng giá	\$bucket	2.2.3
YC13	Tìm NSX bán laptop nhưng không bán PC; NSX có từ hai loại sản phẩm	\$addToSet, \$all, \$nin, \$size	2.2.3
YC14	Tổng hợp danh mục laptop của từng NSX từ collection makers	\$lookup, \$filter	2.3.1
YC15	Bổ sung quy mô danh mục của NSX cho từng máy in	\$lookup, \$unwind	2.3.1
YC16	Thống kê số máy in màu và giá máy in thấp nhất theo NSX	\$lookup (let, pipeline), \$expr	2.3.1

2.1.2. Thực hiện và phân tích kết quả truy vấn

YC1 – Laptop giá dưới 1.000 USD, sắp xếp theo giá tăng dần

Yêu cầu phục vụ tư vấn khách hàng ở phân khúc giá thấp. Điều kiện lọc kết hợp loại sản phẩm và giá; phép chiếu lấy các thông số cấu hình chính trong specs:

```
db.products.find(
  { type: "laptop", price: { $lt: 1000 } },
  { _id: 0, maker: 1, model: 1, "specs.speed": 1, "specs.ram": 1, "specs.hd": 1,
    price: 1 }
).sort({ price: 1 })
```

Kết quả thực thi gồm 4 document (Bảng 2.2):

Bảng 2.2. Kết quả YC1 – laptop giá dưới 1.000 USD

maker	model	specs.speed	specs.ram	specs.hd	price
E	2003	1.8	512	60	549
F	2009	1.6	512	80	680
F	2008	1.6	1024	100	900
E	2002	1.73	1024	80	949

Có 4 laptop thỏa mãn, rẻ nhất là model 2003 của E (549 USD). Truy vấn tương đương trong SQL cần phép nối hai bảng: `SELECT maker, model, speed, ram, hd, price FROM Product NATURAL JOIN Laptop WHERE price < 1000 ORDER BY price;` – trong mô hình đã chọn, nhờ cấu hình được nhúng vào specs, MongoDB chỉ cần đọc một collection.

YC2 – Các NSX có laptop với ổ cứng từ 100 GB trở lên

Phương thức `distinct` trả về danh sách giá trị không trùng lặp của một trường; kết quả gồm 5 NSX (ứng với các laptop 2005, 2007, 2001, 2008, 2010):

```
db.products.distinct("maker", { type: "laptop", "specs.hd": { $gte: 100 } })
```

```
[ 'A', 'B', 'E', 'F', 'G' ]
```

YC3 – Laptop màn hình từ 15,4 inch và RAM từ 1024 MB

Hai điều kiện trên hai trường của embedded document được kết hợp ngầm định theo phép AND; kết quả sắp xếp giảm dần theo kích thước màn hình, sau đó theo giá:

```
db.products.find(
  { type: "laptop", "specs.screen": { $gte: 15.4 }, "specs.ram": { $gte: 1024 } },
  { _id: 0, maker: 1, model: 1, "specs.ram": 1, "specs.screen": 1, price: 1 }
).sort({ "specs.screen": -1, price: -1 })
```

Kết quả thực thi gồm 6 document (Bảng 2.3):

Bảng 2.3. Kết quả YC3 – laptop màn hình từ 15,4 inch và RAM từ 1024 MB

maker	model	specs.ram	specs.screen	price
E	2001	2048	20.1	3673
A	2005	1024	17	2500
E	2002	1024	17	949
G	2010	2048	15.4	2300
A	2006	2048	15.4	1700
F	2008	1024	15.4	900

Laptop 2001 của E có màn hình lớn nhất (20,1 inch) và giá cao nhất; khi cùng kích thước màn hình, giá giảm dần quyết định thứ tự.

YC4 – Sản phẩm của NSX E có giá từ 200 đến 1.000 USD

Điều kiện khoảng được biểu diễn bằng hai toán tử \$gte và \$lte trên cùng một trường; kết quả gồm 4 document (kiểm tra lại bằng countDocuments cũng trả về 4):

```
db.products.find(
  { maker: "E", price: { $gte: 200, $lte: 1000 } },
  { _id: 0, model: 1, type: 1, price: 1 }
).sort({ price: 1 })
```

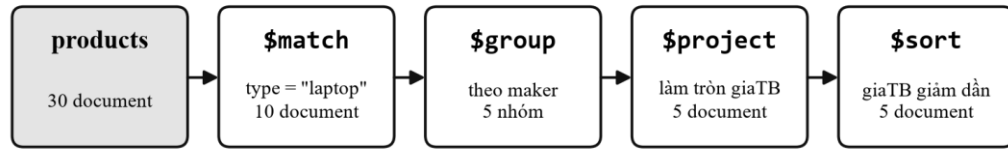
```
[
  { model: 3002, type: 'printer', price: 239 },
  { model: 2003, type: 'laptop', price: 549 },
  { model: 3003, type: 'printer', price: 899 },
  { model: 2002, type: 'laptop', price: 949 }
]
```

Các PC của E không xuất hiện do không có trường price – điều kiện so sánh trên trường không tồn tại cho kết quả sai, đúng ngữ nghĩa nghiệp vụ.

2.2. Tổng hợp dữ liệu bằng Aggregation Pipeline

Aggregation Pipeline là cơ chế tổng hợp dữ liệu của MongoDB, trong đó dữ liệu đi qua một chuỗi các giai đoạn (stage); đầu ra của stage trước là đầu vào của stage sau (Hình 2.1). Mỗi stage thực hiện một phép biến đổi [3], [7]: \$match lọc document (tương đương WHERE/HAVING), \$project chọn và tính trường mới (SELECT), \$group nhóm và tính giá trị tổng hợp (GROUP BY), \$sort/\$limit sắp

xếp, giới hạn (ORDER BY/LIMIT), \$bucket phân nhóm theo khoảng, \$unwind tách mảng, \$lookup kết hợp collection (LEFT OUTER JOIN) và \$out ghi kết quả ra collection.



Dữ liệu đi qua lần lượt từng stage; đầu ra của stage trước là đầu vào của stage sau

Hình 2.1. Luồng xử lý Aggregation Pipeline của yêu cầu YC8

2.2.1. Lọc và lựa chọn dữ liệu với \$match, \$project

YC5 – Báo cáo cấu hình laptop

Stage \$match lọc laptop; stage \$project đưa các trường lồng trong specs lên cấp gốc với tên tiếng Việt, quy đổi RAM từ MB sang GB bằng \$divide và phân loại hiệu năng bằng biểu thức điều kiện \$cond (tốc độ từ 2,0 GHz là “Hiệu năng cao”):

```
db.products.aggregate([
  { $match: { type: "laptop" } },
  { $project: {
    _id: 0, model: 1, maker: 1,
    tocDo: "$specs.speed",
    ramGB: { $divide: ["$specs.ram", 1024] },
    oCung: "$specs.hd",
    manHinh: "$specs.screen",
    phanLoai: { $cond: [ { $gte: ["$specs.speed", 2.0] }, "Hiệu năng cao",
      "Phổ thông" ] },
    price: 1
  } },
  { $sort: { model: 1 } }
])
```

Kết quả thực thi gồm 10 document, được trình bày lại dưới dạng bảng trong Bảng 2.4.

Bảng 2.4. Kết quả YC5 – báo cáo cấu hình laptop

model	maker	tocDo	ramGB	oCung	manHinh	price	phanLoai
2001	E	2	2	240	20.1	3673	Hiệu năng cao
2002	E	1.73	1	80	17	949	Phổ thông
2003	E	1.8	0.5	60	15.4	549	Phổ thông
2004	A	2	0.5	60	13.3	1150	Hiệu năng cao
2005	A	2.16	1	120	17	2500	Hiệu năng cao
2006	A	2	2	80	15.4	1700	Hiệu năng cao
2007	B	1.83	1	120	13.3	1429	Phổ thông
2008	F	1.6	1	100	15.4	900	Phổ thông
2009	F	1.6	0.5	80	14.1	680	Phổ thông

model	maker	tocDo	ramGB	oCung	manHinh	price	phanLoai
2010	G	2	2	160	15.4	2300	Hiệu năng cao

Năm laptop thuộc nhóm “Hiệu năng cao” (2001, 2004, 2005, 2006, 2010); RAM chỉ gồm ba mức 0,5 GB, 1 GB và 2 GB. Phép tính được thực hiện ngay tại máy chủ CSDL.

YC6 – Danh sách máy in kèm nhãn chế độ in

Biểu thức \$cond chuyển giá trị logic specs.color thành nhãn “In màu” / “Đen trắng”; kết quả sắp xếp theo giá:

```
db.products.aggregate([
  { $match: { type: "printer" } },
  { $project: {
    _id: 0, model: 1, maker: 1,
    congNghe: "$specs.type",
    mauIn: { $cond: [ "$specs.color", "In màu", "Đen trắng" ] },
    price: 1
  } },
  { $sort: { price: 1, model: 1 } }
])
```

Kết quả gồm 7 máy in theo thứ tự giá tăng dần: 3001 (E, ink-jet, In màu, 99), 3006 (H, ink-jet, In màu, 100), 3004 (D, ink-jet, In màu, 120), 3005 (D, laser, Đen trắng, 120), 3007 (H, laser, In màu, 200), 3002 (E, laser, Đen trắng, 239), 3003 (E, laser, In màu, 899). Máy in phun mực đều in màu và có giá thấp (99 – 120 USD); máy in laser có cả loại màu và đen trắng với dải giá rộng hơn (120 – 899 USD).

2.2.2. Nhóm và tính toán với \$group, \$sum, \$avg

YC7 – Số sản phẩm theo NSX và loại

Khóa nhóm _id là một document ghép gồm maker và type; \$sum: 1 đếm số document trong mỗi nhóm:

```
db.products.aggregate([
  { $group: {
    _id: { maker: "$maker", type: "$type" },
    soLuong: { $sum: 1 }
  } },
  { $sort: { "_id.maker": 1, "_id.type": 1 } }
])
```

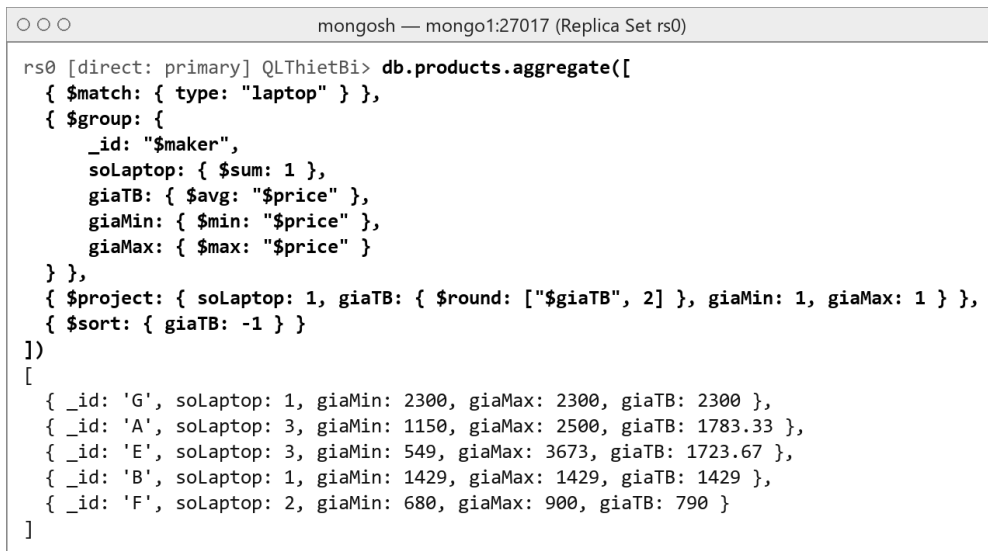
```
[
  { _id: { maker: 'A', type: 'laptop' }, soLuong: 3 },
  { _id: { maker: 'A', type: 'pc' }, soLuong: 3 },
  { _id: { maker: 'B', type: 'laptop' }, soLuong: 1 },
  { _id: { maker: 'B', type: 'pc' }, soLuong: 3 },
  ...
  { _id: { maker: 'H', type: 'printer' }, soLuong: 2 }
]
```

Kết quả gồm 13 nhóm (NSX, loại), trùng khớp với Bảng 1.4.

YC8 – Thống kê giá laptop theo NSX

Pipeline gồm bốn stage như Hình 2.1: lọc laptop, nhóm theo maker và tính số lượng, giá trung bình, giá thấp nhất, cao nhất; làm tròn giá trung bình đến hai chữ số thập phân bằng \$round; sắp xếp giảm dần theo giá trung bình. Lệnh và kết quả thực thi được thể hiện trong Hình 2.2.

```
db.products.aggregate([
  { $match: { type: "laptop" } },
  { $group: {
    _id: "$maker",
    soLaptop: { $sum: 1 },
    giaTB: { $avg: "$price" },
    giaMin: { $min: "$price" },
    giaMax: { $max: "$price" }
  } },
  { $project: { soLaptop: 1, giaTB: { $round: ["$giaTB", 2] }, giaMin: 1, giaMax: 1 } },
  { $sort: { giaTB: -1 } }
])
```



```
mongosh — mongo1:27017 (Replica Set rs0)

rs0 [direct: primary] QLThietBi> db.products.aggregate([
  { $match: { type: "laptop" } },
  { $group: {
    _id: "$maker",
    soLaptop: { $sum: 1 },
    giaTB: { $avg: "$price" },
    giaMin: { $min: "$price" },
    giaMax: { $max: "$price" }
  } },
  { $project: { soLaptop: 1, giaTB: { $round: ["$giaTB", 2] }, giaMin: 1, giaMax: 1 } },
  { $sort: { giaTB: -1 } }
])
[
  { _id: 'G', soLaptop: 1, giaMin: 2300, giaMax: 2300, giaTB: 2300 },
  { _id: 'A', soLaptop: 3, giaMin: 1150, giaMax: 2500, giaTB: 1783.33 },
  { _id: 'E', soLaptop: 3, giaMin: 549, giaMax: 3673, giaTB: 1723.67 },
  { _id: 'B', soLaptop: 1, giaMin: 1429, giaMax: 1429, giaTB: 1429 },
  { _id: 'F', soLaptop: 2, giaMin: 680, giaMax: 900, giaTB: 790 }
]
```

Hình 2.2. Kết quả YC8 – thống kê giá laptop theo nhà sản xuất

G có giá trung bình cao nhất (2300 USD) nhưng chỉ có một laptop; trong các NSX có nhiều laptop, A cao nhất (1783,33 USD), E có biên độ giá rộng nhất (549 – 3673 USD), F thấp nhất (790 USD).

YC9 – Thống kê theo loại sản phẩm

Biểu thức \$cond kết hợp \$ifNull đếm số sản phẩm có thông tin giá trong từng loại; \$sum và \$avg tự động bỏ qua các document không có trường price:

```
db.products.aggregate([
  { $group: {
    _id: "$type",
    soLuong: { $sum: 1 },
    coGia: { $sum: { $cond: [ { $ifNull: ["$price", false] }, 1, 0 ] } },
    tongGia: { $sum: "$price" },
    giaTB: { $avg: "$price" }
  } },
  { $project: { soLuong: 1, coGia: 1, tongGia: 1, giaTB: { $round: ["$giaTB", 2] } } },
  { $sort: { soLuong: -1 } }
])
```

```
[
  { _id: 'pc', soLuong: 13, coGia: 0, tongGia: 0, giaTB: null },
  { _id: 'laptop', soLuong: 10, coGia: 10, tongGia: 15830, giaTB: 1583 },
  { _id: 'printer', soLuong: 7, coGia: 7, tongGia: 1777, giaTB: 253.86 }
]
```

Với pc, tổng giá bằng 0 và giá trung bình null do không có sản phẩm nào có giá – đúng với dữ liệu nguồn. Giá trung bình laptop (1583 USD) gấp khoảng 6,2 lần máy in (253,86 USD).

YC10 – Dung lượng ổ cứng xuất hiện ở từ hai laptop trở lên

Stage \$match đặt **sau** \$group đóng vai trò như mệnh đề HAVING trong SQL; kết quả gồm ba mức 80 GB (3 laptop), 60 GB và 120 GB (mỗi mức 2 laptop):

```
db.products.aggregate([
  { $match: { type: "laptop" } },
  { $group: { _id: "$specs.hd", soLaptop: { $sum: 1 }, dsModel: { $push: "$model" } } },
  { $match: { soLaptop: { $gte: 2 } } },
  { $sort: { soLaptop: -1, _id: 1 } }
])
```

```
[
  { _id: 80, soLaptop: 3, dsModel: [ 2006, 2002, 2009 ] },
  { _id: 60, soLaptop: 2, dsModel: [ 2004, 2003 ] },
  { _id: 120, soLaptop: 2, dsModel: [ 2005, 2007 ] }
]
```

2.2.3. Sắp xếp và kết hợp các stage trong Pipeline

YC11 – Ba sản phẩm có giá cao nhất

Kết hợp \$sort giảm dần theo giá với \$limit: 3; MongoDB tối ưu cặp stage này thành thuật toán top-k, chỉ giữ 3 phần tử trong bộ nhớ khi sắp xếp; ba sản phẩm đắt nhất đều là laptop:

```
db.products.aggregate([
  { $match: { price: { $exists: true } } },
  { $sort: { price: -1 } },
  { $limit: 3 },
  { $project: { _id: 0, maker: 1, model: 1, type: 1, price: 1 } }
])
```

```
[
  { maker: 'E', model: 2001, type: 'laptop', price: 3673 },
  { maker: 'A', model: 2005, type: 'laptop', price: 2500 },
  { maker: 'G', model: 2010, type: 'laptop', price: 2300 }
]
```

YC12 – Phân khúc laptop theo khoảng giá

Stage \$bucket phân nhóm theo các mốc boundaries [0, 1000), [1000, 2000), [2000, 5000); _id của mỗi nhóm là cận dưới của khoảng:

```
db.products.aggregate([
  { $match: { type: "laptop" } },
  { $bucket: {
    groupBy: "$price",
    boundaries: [0, 1000, 2000, 5000],
    default: "Khác",
    output: { soLaptop: { $sum: 1 }, dsModel: { $push: "$model" },
    giaTB: { $avg: "$price" } }
  }
])
```

```

    } },
    { $project: { soLaptop: 1, dsModel: 1, giaTB: { $round: ["$giaTB", 2] } } }
  ])

```

Kết quả thực thi gồm 3 nhóm (Bảng 2.5):

Bảng 2.5. Kết quả YC12 – phân khúc laptop theo giá

_id (cận dưới)	Khoảng giá	soLaptop	dsModel	giaTB
0	[0, 1000)	4	2002, 2003, 2008, 2009	769.5
1000	[1000, 2000)	3	2004, 2006, 2007	1426.33
2000	[2000, 5000)	3	2005, 2001, 2010	2824.33

Phân khúc dưới 1.000 USD có 4 laptop, hai phân khúc còn lại mỗi phân khúc 3 laptop – danh mục phân bố khá cân đối.

YC13 – Phân tích danh mục loại sản phẩm của NSX

(a) NSX bán laptop nhưng không bán PC: nhóm theo maker, gom tập các loại bằng \$addToSet, sau đó lọc tập chứa “laptop” (\$all) và không chứa “pc” (\$nin):

```

db.products.aggregate([
  { $group: { _id: "$maker", cacLoai: { $addToSet: "$type" } } },
  { $match: { cacLoai: { $all: ["laptop"], $nin: ["pc"] } } },
  { $sort: { _id: 1 } }
])

```

```

[
  { _id: 'F', cacLoai: [ 'laptop' ] },
  { _id: 'G', cacLoai: [ 'laptop' ] }
]

```

(b) NSX có từ hai loại sản phẩm trở lên – sử dụng \$size để đếm số phần tử của tập loại và \$sortArray để sắp xếp tập cho dễ đọc:

```

db.products.aggregate([
  { $group: { _id: "$maker", cacLoai: { $addToSet: "$type" } } },
  { $project: { soLoai: { $size: "$cacLoai" },
    cacLoai: { $sortArray: { input: "$cacLoai", sortBy: 1 } } } },
  { $match: { soLoai: { $gte: 2 } } },
  { $sort: { soLoai: -1, _id: 1 } }
])

```

```

[
  { _id: 'E', soLoai: 3, cacLoai: [ 'laptop', 'pc', 'printer' ] },
  { _id: 'A', soLoai: 2, cacLoai: [ 'laptop', 'pc' ] },
  { _id: 'B', soLoai: 2, cacLoai: [ 'laptop', 'pc' ] },
  { _id: 'D', soLoai: 2, cacLoai: [ 'pc', 'printer' ] }
]

```

Truy vấn (a) tương ứng phép trừ tập hợp trong SQL (SELECT maker FROM Product WHERE type = 'laptop' EXCEPT SELECT maker FROM Product WHERE type = 'pc') và cho kết quả F, G. Truy vấn (b) cho thấy E là NSX duy nhất có đủ ba loại; A, B, D có hai loại; C, F, G, H chỉ có một loại.

2.3. Kết hợp và đánh giá kết quả khai thác dữ liệu

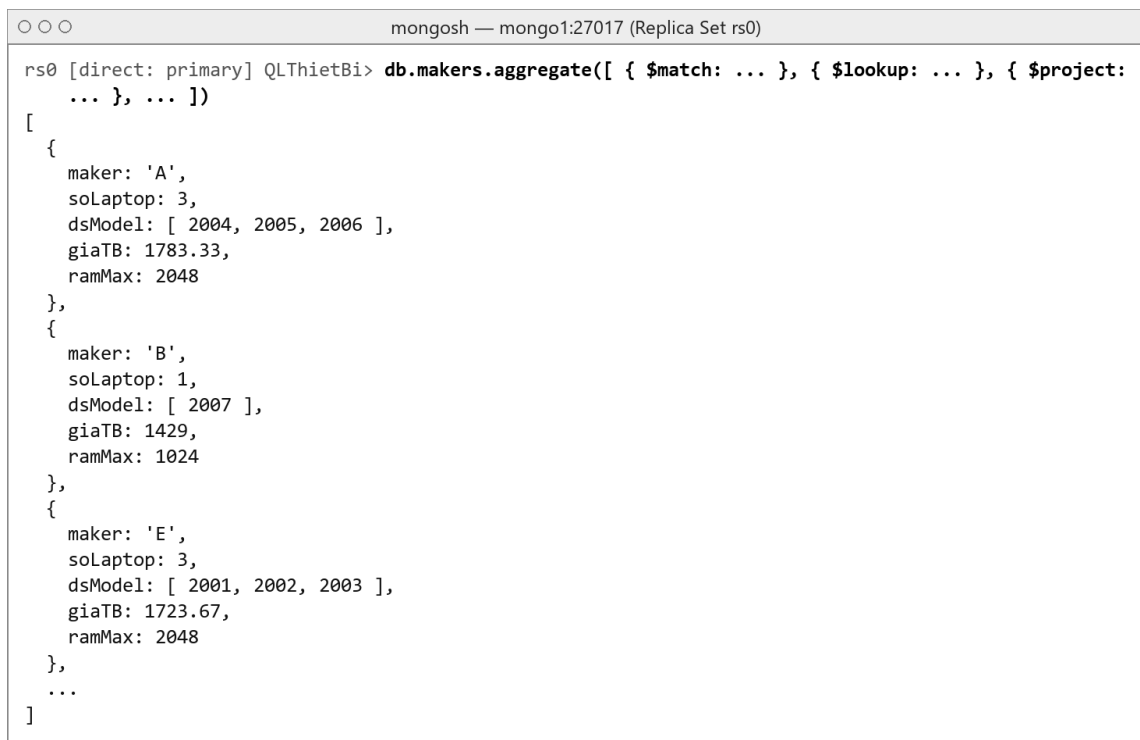
2.3.1. Kết hợp các collection bằng \$lookup

Stage \$lookup thực hiện phép kết nối ngoài trái (left outer join) giữa collection đang xử lý và một collection khác trong cùng cơ sở dữ liệu. MongoDB hỗ trợ hai dạng: dạng so khớp đẳng thức (localField/foreignField) và dạng có pipeline con (let/pipeline) cho phép điều kiện kết nối và xử lý phức tạp hơn. Khi localField là mảng, mỗi phần tử của mảng được so khớp với foreignField.

YC14 – Danh mục laptop của từng NSX (makers → products)

Từ makers, kết nối với products qua products.model (mảng) và model; \$filter chỉ giữ các sản phẩm là laptop; sau đó tính số laptop, danh sách model, giá trung bình và RAM lớn nhất:

Kết quả thực thi được thể hiện trong Hình 2.3 (lược bớt hai document cuối của F và G). Số laptop và giá trung bình của từng NSX trùng khớp với kết quả YC8 được tính trực tiếp trên products, cho thấy mảng tham chiếu trong makers nhất quán với dữ liệu products.



```
rs0 [direct: primary] QLThietBi> db.makers.aggregate([ { $match: ... }, { $lookup: ... }, { $project: ... }, ... ])
[
  {
    maker: 'A',
    soLaptop: 3,
    dsModel: [ 2004, 2005, 2006 ],
    giaTB: 1783.33,
    ramMax: 2048
  },
  {
    maker: 'B',
    soLaptop: 1,
    dsModel: [ 2007 ],
    giaTB: 1429,
    ramMax: 1024
  },
  {
    maker: 'E',
    soLaptop: 3,
    dsModel: [ 2001, 2002, 2003 ],
    giaTB: 1723.67,
    ramMax: 2048
  },
  ...
]
```

Hình 2.3. Kết quả YC14 – kết hợp makers và products bằng \$lookup

Lệnh đầy đủ của pipeline YC14:

```
db.makers.aggregate([
  { $match: { "products.type": "laptop" } },
  { $lookup: {
    from: "products",
    localField: "products.model",
    foreignField: "model",
    as: "chiTiet"
  } }
])
```

```

    } },
    { $project: {
      _id: 0, maker: "$_id",
      laptops: { $filter: { input: "$chiTiet", as: "p", cond: { $eq: ["$$p.type",
        "laptop"] } } }
    } },
    { $project: {
      maker: 1,
      soLaptop: { $size: "$laptops" },
      dsModel: { $sortBy: { input: "$laptops.model", sortBy: 1 } },
      giaTB: { $round: [ { $avg: "$laptops.price" }, 2 ] },
      ramMax: { $max: "$laptops.specs.ram" }
    } },
    { $sort: { maker: 1 } }
  ] )

```

YC15 – Quy mô danh mục của NSX cho từng máy in (products → makers)

Chiều kết nối ngược lại: từ mỗi máy in trong products, tra cứu document NSX tương ứng trong makers qua maker = _id; \$unwind chuyển mảng kết quả một phần tử thành document con:

```

db.products.aggregate([
  { $match: { type: "printer" } },
  { $lookup: { from: "makers", localField: "maker", foreignField: "_id", as: "nsx" } },
  { $unwind: "$nsx" },
  { $project: {
    _id: 0, model: 1, maker: 1, price: 1,
    tongSanPhamCuaNSX: { $size: "$nsx.products" }
  } },
  { $sort: { maker: 1, model: 1 } }
] )

```

Mỗi máy in được bổ sung tổng số sản phẩm của NSX: máy in của D nhận 5, của E nhận 9 (danh mục lớn nhất), của H nhận 2 – H chỉ chuyên về máy in.

YC16 – Thống kê máy in theo NSX bằng \$lookup có pipeline con

Dạng \$lookup với let và pipeline cho phép thực hiện lọc và nhóm ngay trong collection được kết nối. Biến \$\$mk mang giá trị _id của NSX đang xét; \$expr dùng để so sánh trường với biến:

```

db.makers.aggregate([
  { $match: { "products.type": "printer" } },
  { $lookup: {
    from: "products",
    let: { mk: "$_id" },
    pipeline: [
      { $match: { $expr: { $and: [ { $eq: ["$maker", "$$mk"] }, { $eq: ["$type",
        "printer"] } ] } } },
      { $group: {
        _id: null,
        soMayIn: { $sum: 1 },
        soMayInMau: { $sum: { $cond: ["$specs.color", 1, 0] } },
        giaThapNhat: { $min: "$price" }
      } }
    ]
  },
  as: "thongKe"
} ],
{ $unwind: "$thongKe" },
{ $project: {
  _id: 0, maker: "$_id",
  soMayIn: "$thongKe.soMayIn",
  soMayInMau: "$thongKe.soMayInMau",
  giaThapNhat: "$thongKe.giaThapNhat"
} }

```

```
} }  
1)
```

```
[  
  { maker: 'D', soMayIn: 2, soMayInMau: 1, giaThapNhat: 120 },  
  { maker: 'E', soMayIn: 3, soMayInMau: 2, giaThapNhat: 99 },  
  { maker: 'H', soMayIn: 2, soMayInMau: 2, giaThapNhat: 100 }  
]
```

E có 3 máy in (2 máy in màu, giá thấp nhất 99 USD), H có 2 máy in màu, D có 2 máy in (1 máy in màu); tổng 7 máy in khớp dữ liệu nguồn.

2.3.2. Phân tích và đánh giá kết quả theo yêu cầu nghiệp vụ

Toàn bộ 16 yêu cầu ở Bảng 2.1 được thực hiện thành công. Về **tính đúng đắn**, mọi kết quả đều được đối chiếu với dữ liệu nguồn hoặc một truy vấn độc lập: YC1 – YC6, YC10, YC11 đối chiếu với Bảng 1.2, Bảng 1.3; YC7 trùng Bảng 1.4; YC8 trùng YC14; YC9 có tổng giá 15830 / 1777 bằng tổng cột price; YC12 có tổng 10 laptop; YC13 tương đương truy vấn SQL; YC16 có tổng 7 máy in – không phát hiện sai lệch.

- **Hiệu quả của mô hình nhúng:** 13/16 yêu cầu (YC1 – YC13) được trả lời chỉ trên products, không cần phép nối, nhờ cấu hình đã nhúng trong specs – minh chứng cho lựa chọn ở mục 1.2.3.
- **Vai trò của mô hình tham chiếu:** \$lookup chỉ cần khi khai thác theo góc nhìn NSX (YC14 – YC16); mảng tham chiếu trong makers cho phép kết nối nhiều phần tử trong một stage.
- **Khả năng của Aggregation Pipeline:** biểu diễn được các phép tương đương WHERE, GROUP BY, HAVING, ORDER BY, CASE, EXCEPT, JOIN của SQL. Với dữ liệu lớn cần bổ sung chỉ mục (type, price, specs.ram) và đặt \$match ở đầu pipeline.

Kết luận Chương 2

Chương 2 đã hiện thực thành công 16 yêu cầu khai thác theo nghiệp vụ bằng find, distinct, Aggregation Pipeline (\$match, \$project, \$group, \$sort, \$limit, \$bucket) và \$lookup ở cả hai dạng. Kết quả được đối chiếu chéo, trùng khớp với dữ liệu nguồn và khẳng định tính phù hợp của mô hình dữ liệu đã chọn – làm cơ sở để triển khai trên môi trường phân tán ở Chương 3.

CHƯƠNG 3. TRIỂN KHAI CƠ SỞ DỮ LIỆU PHÂN TÁN TRONG MONGODB

Chương 3 triển khai cơ sở dữ liệu QLThietBi trên môi trường phân tán theo hướng 2 của đề tài. Phần đầu chương trình bày cơ chế sao chép dữ liệu bằng Replica Set: cấu hình cụm ba nút, kiểm tra vai trò Primary/Secondary, sự đồng bộ dữ liệu và khả năng chuyển đổi dự phòng khi nút Primary gặp sự cố. Phần tiếp theo phân tích phương án phân bố dữ liệu, lựa chọn khóa phân mảnh và xây dựng Sharded Cluster gồm hai shard. Cuối chương kiểm tra sự sao chép trong từng shard, sự phân bố dữ liệu trên các shard và đánh giá kết quả quản trị dữ liệu.

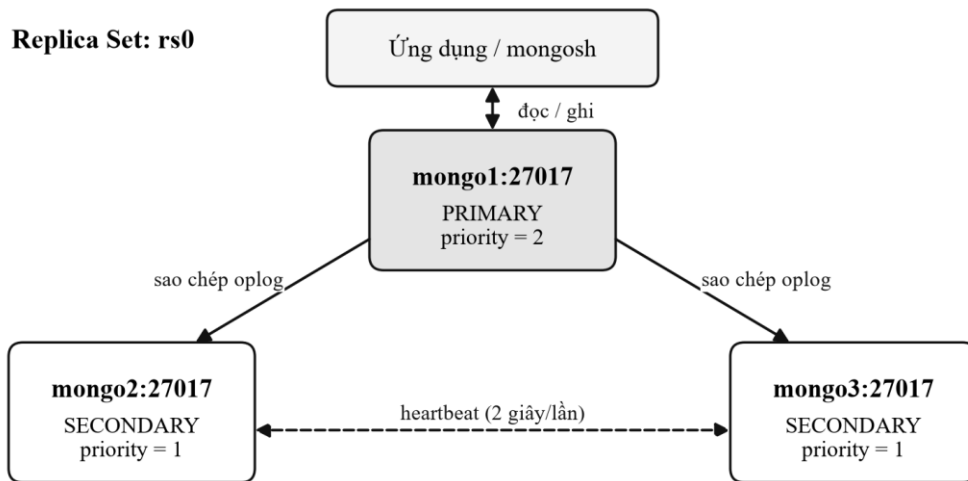
3.1. Xây dựng và kiểm tra Replica Set

3.1.1. Cấu hình Replica Set

Replica Set là một nhóm các tiến trình mongod cùng duy trì một tập dữ liệu, cung cấp khả năng dự phòng (redundancy) và tính sẵn sàng cao (high availability) [4], [7]. Trong một Replica Set, tại mỗi thời điểm chỉ có một nút **Primary** nhận toàn bộ thao tác ghi; mọi thay đổi được ghi vào **oplog** – một capped collection `local.oplog.rs` lưu nhật ký các thao tác theo dạng lũy đẳng. Các nút **Secondary** liên tục đọc oplog từ nguồn đồng bộ (thường là Primary) và áp dụng lại các thao tác, nhờ đó duy trì bản sao dữ liệu giống Primary.

Các nút trao đổi tín hiệu heartbeat với nhau mỗi 2 giây. Nếu Primary không phản hồi quá thời gian `electionTimeoutMillis` (mặc định 10 giây), các nút còn lại tổ chức bầu chọn; nút nhận được đa số phiếu (quá bán số thành viên có quyền bầu) trở thành Primary mới. Thuộc tính `priority` cho phép ưu tiên một nút làm Primary: khi nút có `priority` cao hơn trở lại và đã bắt kịp dữ liệu, nó sẽ yêu cầu bầu chọn để giành lại vai trò Primary. Ứng dụng điều chỉnh mức độ bảo đảm khi ghi bằng **write concern** (ví dụ `w: 1`, `w: "majority"`, `w: 3`) và nơi đọc dữ liệu bằng **read preference** (`primary`, `secondary`, `nearest`, ...).

Replica Set `rs0` của tiểu luận gồm ba nút `mongo1`, `mongo2`, `mongo3` (cổng 27017) chạy trong ba container Docker trên cùng mạng `mongo-net` (Hình 3.1); mỗi nút có một phiếu bầu, nút `mongo1` được đặt `priority 2` để là Primary ưu tiên.



Hình 3.1. Kiến trúc Replica Set rs0 gồm ba nút

Mỗi nút được khởi động với tham số `--replSet rs0`. Trích đoạn tệp `docker-compose.yml` khai báo nút `mongo1` (hai nút còn lại tương tự):

```

services:
  mongo1:
    image: mongo:8.0
    container_name: mongo1
    hostname: mongo1
    networks: [mongo-net]
    command: mongod --replSet rs0 --bind_ip_all --port 27017
              --wiredTigerCacheSizeGB 0.25
  
```

Sau khi các container khởi động, kết nối mongosh tới `mongo1` và khởi tạo Replica Set bằng `rs.initiate()` với cấu hình ba thành viên:

```

rs.initiate({
  _id: "rs0",
  members: [
    { _id: 0, host: "mongo1:27017", priority: 2 },
    { _id: 1, host: "mongo2:27017", priority: 1 },
    { _id: 2, host: "mongo3:27017", priority: 1 }
  ]
})
  
```

```

{
  ok: 1,
  ...
}
  
```

Kết quả `ok: 1` cho biết Replica Set đã được khởi tạo; lệnh `rs.conf()` xác nhận ba thành viên với priority lần lượt 2, 1, 1 và mỗi thành viên một phiếu bầu.

3.1.2. Kiểm tra Primary, Secondary và sao chép dữ liệu

a) Trạng thái các nút

Lệnh `rs.status()` cho biết trạng thái của từng thành viên (Hình 3.2): `mongo1` là PRIMARY, `mongo2` và `mongo3` là SECONDARY với nguồn đồng bộ `syncSourceHost: mongo1:27017`; cả ba nút đều khỏe (`health: 1`) và có cùng thời điểm thao tác cuối (`optimeDate`), tức là các Secondary đã bắt kịp Primary.

```

○ ○ ○ mongosh — mongo1:27017 (Replica Set rs0)

rs0 [direct: primary] QLThietBi> rs.status().members.map(m => ({
  name: m.name, stateStr: m.stateStr, health: m.health,
  syncSourceHost: m.syncSourceHost, optimeDate: m.optimeDate
}))
[
  {
    name: 'mongo1:27017',
    stateStr: 'PRIMARY',
    health: 1,
    syncSourceHost: '',
    optimeDate: ISODate('2026-10-05T07:33:48.000Z')
  },
  {
    name: 'mongo2:27017',
    stateStr: 'SECONDARY',
    health: 1,
    syncSourceHost: 'mongo1:27017',
    optimeDate: ISODate('2026-10-05T07:33:48.000Z')
  },
  {
    name: 'mongo3:27017',
    stateStr: 'SECONDARY',
    health: 1,
    syncSourceHost: 'mongo1:27017',
    optimeDate: ISODate('2026-10-05T07:33:48.000Z')
  }
]

```

Hình 3.2. Trạng thái các thành viên của Replica Set rs0

b) Ghi dữ liệu tại Primary và theo dõi oplog

Để kiểm tra cơ chế sao chép mà không tác động tới dữ liệu nguồn, một collection tạm `kiemtra_saochep` được sử dụng (sẽ bị xóa sau khi kiểm tra). Ghi một document với write concern `w: 3` – thao tác chỉ được xác nhận khi cả ba nút đã ghi nhận:

```

db.kiemtra_saochep.insertOne(
  { noiDung: "Ghi tại Primary mongo1", thoiDiem: new Date() },
  { writeConcern: { w: 3, wtimeout: 5000 } }
)

```

```

{
  acknowledged: true,
  insertedId: ObjectId('6ac352df06e04c3e50b8c69a')
}

```

Thao tác ghi được lưu trong oplog của Primary dưới dạng một bản ghi op: "i" (insert) kèm nội dung document và dấu thời gian `ts`:

```

db.getSiblingDB("local").oplog.rs.find(
  { ns: "QLThietBi.kiemtra_saochep", op: "i" },
  { _id: 0, op: 1, ns: 1, o: 1, ts: 1 }
).sort({ $natural: -1 }).limit(1)

```

```

[
  {
    op: 'i',
    ns: 'QLThietBi.kiemtra_saochep',
    ...
    noiDung: 'Ghi tại Primary mongo1',
    ...
  },
  ts: Timestamp({ t: 1791185631, i: 2 })
]

```

Độ trễ sao chép của các Secondary so với Primary:

```
rs.printSecondaryReplicationInfo()
```

```
source: mongo2:27017
...
replLag: '0 secs (0 hrs) behind the primary '
...
source: mongo3:27017
...
replLag: '0 secs (0 hrs) behind the primary '
```

Cả hai Secondary đều có độ trễ 0 giây, đồng bộ tới cùng thời điểm với Primary.

c) Đọc dữ liệu tại Secondary

Kết nối trực tiếp tới mongo2 (Hình 3.3): nút này xác nhận vai trò Secondary (`isWritablePrimary: false`); số document của `products` (30) và `makers` (8) giống hệt Primary; document vừa ghi tại Primary đã có mặt. Thao tác ghi trực tiếp vào Secondary bị từ chối với lỗi `NotWritablePrimary` – bảo đảm mọi thao tác ghi chỉ đi qua Primary.

```

○○○ mongosh — mongo2:27017 (Replica Set rs0)

rs0 [direct: secondary] QLThietBi> ({ me, isWritablePrimary, secondary, primary }) => ({ me,
  isWritablePrimary, secondary, primary })(db.hello())
{
  me: 'mongo2:27017',
  isWritablePrimary: false,
  secondary: true,
  primary: 'mongo1:27017'
}

rs0 [direct: secondary] QLThietBi> ({ products: db.products.countDocuments(), makers:
  db.makers.countDocuments() })
{ products: 30, makers: 8 }

rs0 [direct: secondary] QLThietBi> db.kiemtra_saochep.find({}, { _id: 0, noiDung: 1 })
[ { noiDung: 'Ghi tại Primary mongo1' } ]

rs0 [direct: secondary] QLThietBi> db.products.insertOne({ maker: "X", model: 9002, type: "pc" })
Uncaught
MongoServerError[NotWritablePrimary]: not primary

```

Hình 3.3. Đọc dữ liệu và thử ghi tại nút Secondary mongo2

d) Kiểm tra sự đồng bộ bằng giá trị băm dữ liệu

Để kiểm chứng chặt chẽ rằng dữ liệu trên các nút **giống hệt nhau**, lệnh `dbHash` được thực hiện trên từng nút; lệnh này tính giá trị băm MD5 trên toàn bộ nội dung của từng collection:

```
db.runCommand({ dbHash: 1, collections: ["products", "makers"] }).collections
```

Bảng 3.1. Giá trị băm dữ liệu (dbHash) trên các nút của rs0

Nút	Vai trò	products	makers
mongo1	Primary	ec7a6561b8a5c1a2fec3992630658680	d5370fd06a542c1d77000b1c90b07829
mongo2	Secondary	ec7a6561b8a5c1a2fec3992630658680	d5370fd06a542c1d77000b1c90b07829
mongo3	Secondary	ec7a6561b8a5c1a2fec3992630658680	d5370fd06a542c1d77000b1c90b07829

Giá trị băm của cả hai collection trên ba nút trùng nhau hoàn toàn (Bảng 3.1), khẳng định dữ liệu đã được sao chép đầy đủ và nhất quán.

e) Kiểm thử chuyển đổi dự phòng (failover)

Kịch bản kiểm thử mô phỏng sự cố nút Primary gồm hai giai đoạn: (b) dừng đột ngột container mongo1 để các nút còn lại bầu Primary mới; (c) khởi động lại mongo1 để nút này bắt kịp dữ liệu và giành lại vai trò Primary nhờ priority cao hơn. Kết quả giai đoạn (b) được thể hiện trong Hình 3.4: sau khi dừng mongo1, rs.status() (kết nối qua chuỗi kết nối Replica Set tới mongo2, mongo3) cho thấy mongo1 ở trạng thái “not reachable/healthy”, mongo2 đã được bầu làm PRIMARY. Hệ thống vẫn tiếp nhận thao tác ghi với write concern majority (2/3 nút) – dịch vụ không bị gián đoạn.

```

Terminal & mongosh — kiểm thử chuyển đổi dự phòng

$ docker stop mongo1 && sleep 20 && docker ps -a --filter name=mongo1 --format "{{.Names}}: {{.Status}}"
mongo1
mongo1: Exited (137) 21 seconds ago

rs0 [primary] QLThietBi> rs.status().members.map(m => ({ name: m.name, stateStr: m.stateStr, health:
m.health }))
[
  {
    name: 'mongo1:27017',
    stateStr: '(not reachable/healthy)',
    health: 0
  },
  { name: 'mongo2:27017', stateStr: 'PRIMARY', health: 1 },
  { name: 'mongo3:27017', stateStr: 'SECONDARY', health: 1 }
]

rs0 [primary] QLThietBi> db.kiemtra_saochep.insertOne(
  { noiDung: "Ghi trong lúc mongo1 dừng", thoiDiem: new Date() },
  { writeConcern: { w: "majority" } }
)
{
  acknowledged: true,
  insertedId: ObjectId('6ac352ffaa71ff6d05c6f3c5')
}

```

Hình 3.4. Dừng nút Primary mongo1 – Replica Set bầu mongo2 làm Primary mới

Giai đoạn (c) – Hình 3.5: sau khi khởi động lại, mongo1 tự động đồng bộ các thao tác đã diễn ra trong thời gian dừng (document “Ghi trong lúc mongo1 dừng” đã có mặt) và trở lại vai trò PRIMARY; mongo2 trở về SECONDARY.

```

Terminal & mongosh — mongo1 khởi động lại

$ docker start mongo1 && sleep 30 && docker ps --filter name=mongo1 --format "{{.Names}}: {{.Status}}"
mongo1
mongo1: Up 31 seconds

rs0 [direct: primary] QLThietBi> rs.status().members.map(m => ({ name: m.name, stateStr: m.stateStr,
health: m.health })))
[
  { name: 'mongo1:27017', stateStr: 'PRIMARY', health: 1 },
  { name: 'mongo2:27017', stateStr: 'SECONDARY', health: 1 },
  { name: 'mongo3:27017', stateStr: 'SECONDARY', health: 1 }
]

rs0 [direct: primary] QLThietBi> db.kiemtra_saochep.find({}, { _id: 0, noiDung: 1 })
[
  { noiDung: 'Ghi tại Primary mongo1' },
  { noiDung: 'Ghi trong lúc mongo1 dừng' }
]

```

Hình 3.5. Khởi động lại mongo1 – nút tự đồng bộ và giành lại vai trò Primary

Sau kiểm thử, collection tạm được xóa; giá trị băm toàn cơ sở dữ liệu QLThietBi được so sánh lại trên ba nút:

```

db.kiemtra_saochep.drop()
db.runCommand({ dbHash: 1, collections: ["products", "makers"] }).md5

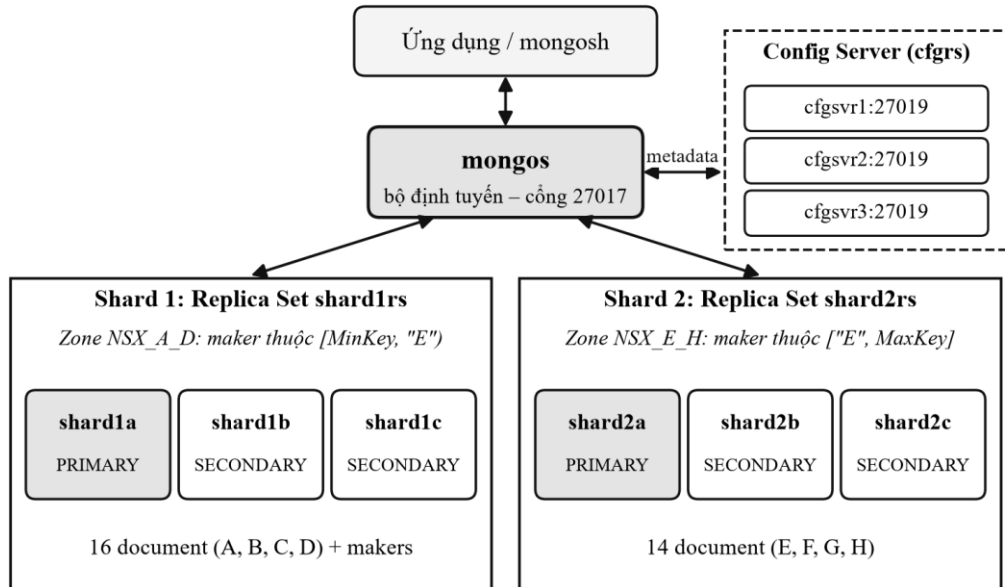
// mongo1
c78ba5b0e9b2d3609853f50f790bf49e
// mongo2
c78ba5b0e9b2d3609853f50f790bf49e
// mongo3
c78ba5b0e9b2d3609853f50f790bf49e

```

Ba nút có cùng giá trị băm, đồng thời trên mongo2 chỉ còn hai collection products và makers – Replica Set đã trở về trạng thái ban đầu, dữ liệu nguồn không bị ảnh hưởng bởi quá trình kiểm thử.

3.2. Xây dựng Sharded Cluster

Sharding là phương pháp phân bố dữ liệu trên nhiều máy chủ để mở rộng theo chiều ngang. Một Sharded Cluster của MongoDB gồm ba thành phần [7]: (1) **shard** – mỗi shard lưu một tập con dữ liệu và được triển khai dưới dạng Replica Set; (2) **mongos** – bộ định tuyến nhận truy vấn từ ứng dụng và chuyển tới các shard liên quan; (3) **config server** – Replica Set lưu siêu dữ liệu của cụm (danh sách shard, phân bố chunk, zone). Dữ liệu của collection được phân mảnh theo **shard key**; mỗi **chunk** là một khoảng liên tục các giá trị shard key, được gán cho một shard. Tiến trình **balancer** di chuyển chunk giữa các shard để cân bằng dữ liệu và bảo đảm rằng buộc **zone**. Kiến trúc cụm được triển khai trong tiểu luận được thể hiện trong Hình 3.6.



Hình 3.6. Kiến trúc Sharded Cluster được triển khai

3.2.1. Phân tích và lựa chọn Shard Key

Shard key quyết định hiệu quả của toàn bộ cụm và rất khó thay đổi về sau, vì vậy cần được lựa chọn dựa trên các tiêu chí [4], [7]:

- **Lực lượng (cardinality):** số giá trị phân biệt của khóa – quyết định số chunk tối đa có thể tạo ra.
- **Tần suất (frequency):** mức độ lặp lại của các giá trị – giá trị xuất hiện quá nhiều dễ tạo chunk quá lớn (jumbo chunk) không thể chia nhỏ.
- **Tính đơn điệu (monotonicity):** khóa tăng dần theo thời gian khiến mọi thao tác chèn dồn vào chunk cuối cùng (điểm nóng ghi).
- **Khả năng định tuyến truy vấn (query isolation):** truy vấn có chứa shard key được mongos chuyển tới đúng shard (targeted query); ngược lại phải gửi tới mọi shard (scatter-gather).

Áp dụng các tiêu chí cho collection products với mẫu truy cập ở Bảng 1.5, năm phương án khóa ứng viên được phân tích trong Bảng 3.2.

Bảng 3.2. Phân tích các phương án shard key cho collection products

Ứng viên	Lực lượng	Tần suất / tính đơn điệu	Định tuyến truy vấn	Đánh giá
{ type: 1 }	3	Lệch: pc chiếm 13/30	Tốt cho lọc theo loại (TC3)	Loại: tối đa 3 chunk
{ maker: 1 }	8	Lệch: E chiếm 9/30	Tốt cho theo NSX (TC2, TC5)	Hạn chế: không chia nhỏ được dữ liệu của một NSX
{ model: 1 }	30 (duy nhất)	Model mới tăng dần → điểm nóng ghi	Chỉ tốt cho tra cứu theo model (TC1)	Loại
{ model: "hashed" }	30 (duy nhất)	Phân bố đều, không đơn điệu	Truy vấn theo NSX, khoảng giá trị phải gửi mọi shard	Phương án so sánh
{ maker: 1, model: 1 }	30 (duy nhất)	Đều ở mức khóa ghép; model mới phân tán theo NSX	Định tuyến được truy vấn theo maker và maker + model	Lựa chọn

Tiểu luận lựa chọn **shard key ghép** `{ maker: 1, model: 1 }` với **phân mảnh theo khoảng (ranged sharding)**. Thành phần maker đứng đầu giúp mọi truy vấn theo NSX (TC2, TC5) và tra cứu theo cặp maker – model được định tuyến tới một shard duy nhất, đồng thời giữ các sản phẩm của cùng NSX nằm gần nhau (data locality). Thành phần model bổ sung bảo đảm khóa có lực lượng bằng số document, cho phép chia nhỏ chunk ngay cả trong phạm vi một NSX khi dữ liệu tăng trưởng – khắc phục hạn chế của khóa { maker: 1 } đơn lẻ.

Về mặt lý thuyết CSDL phân tán, phương án tương ứng với **phân mảnh ngang nguyên thủy** quan hệ products theo các vị từ đơn giản trên thuộc tính maker [5]:

$F1 = \sigma(\text{maker} < 'E')(\text{products})$	→ cấp phát cho shard1rs	(zone NSX_A_D)
$F2 = \sigma(\text{maker} \geq 'E')(\text{products})$	→ cấp phát cho shard2rs	(zone NSX_E_H)

Cách phân mảnh thỏa mãn ba tính chất đúng đắn: **đầy đủ** (mỗi document thuộc một mảnh vì maker luôn có giá trị), **tái thiết được** ($\text{products} = F1 \cup F2$) và **rời nhau** ($F1 \cap F2 = \emptyset$). Theo Bảng 1.4, mảnh F1 (NSX A, B, C, D) có 16 document, mảnh F2 (NSX E, F, G, H) có 14 document – tỷ lệ 53% / 47%, khá cân bằng.

Do tổng dung lượng dữ liệu chỉ khoảng 3 KB, nhỏ hơn rất nhiều kích thước chunk mặc định 128 MB, balancer sẽ không tự chia và di chuyển chunk. Vì vậy, việc cấp phát các mảnh cho shard được khai báo tường minh bằng **zone sharding**: mỗi shard được gán một zone, mỗi zone ứng với một khoảng giá trị shard key. Khi dữ liệu

tăng, các chunk trong một zone vẫn được tự động chia nhỏ và cân bằng giữa các shard thuộc cùng zone.

Collection makers chỉ có 8 document nhỏ nên **không phân mảnh**; nó được lưu trên shard chính (primary shard) của cơ sở dữ liệu là shard1rs. Stage \$lookup giữa makers và collection đã phân mảnh products vẫn được hỗ trợ. Cần lưu ý rằng trên collection đã phân mảnh, chỉ mục duy nhất phải có tiền tố là shard key, vì vậy ràng buộc duy nhất được khai báo trên { maker: 1, model: 1 }.

3.2.2. Cấu hình và thực hiện Sharding

Các thành phần của Sharded Cluster (Hình 3.6) được liệt kê trong Bảng 3.3. Mỗi shard là một Replica Set ba nút, do đó cụm vừa phân mảnh vừa sao chép dữ liệu.

Bảng 3.3. Các thành phần của Sharded Cluster

Thành phần	Replica Set	Container (cổng)	Vai trò
Config Server	cfgrs	cfgsvr1, cfgsvr2, cfgsvr3 (27019)	Lưu metadata: shard, chunk, zone
Shard 1	shard1rs	shard1a, shard1b, shard1c (27018)	Lưu mảnh F1 (NSX A – D) và makers
Shard 2	shard2rs	shard2a, shard2b, shard2c (27018)	Lưu mảnh F2 (NSX E – H)
Router	–	mongos (27017)	Định tuyến truy vấn của ứng dụng

Bước 1: Khởi tạo Config Server Replica Set

Các nút config server được khởi động với tham số --configsvr --replSet cfgrs; Replica Set được khởi tạo với thuộc tính configsvr: true:

```
rs.initiate({
  _id: "cfgrs",
  configsvr: true,
  members: [
    { _id: 0, host: "cfgsvr1:27019" },
    { _id: 1, host: "cfgsvr2:27019" },
    { _id: 2, host: "cfgsvr3:27019" }
  ]
})
```

Bước 2: Khởi tạo các shard Replica Set

Các nút shard được khởi động với tham số `--shardsvr`. Khởi tạo shard1rs (tương tự với shard2rs gồm shard2a, shard2b, shard2c):

```
rs.initiate({
  _id: "shard1rs",
  members: [
    { _id: 0, host: "shard1a:27018", priority: 2 },
    { _id: 1, host: "shard1b:27018" },
    { _id: 2, host: "shard1c:27018" }
  ]
})
```

Bước 3: Thêm shard vào cụm qua mongos

Bộ định tuyến mongos được khởi động với tham số `--configdb cfg1/cfgsvr1:27019,cfgsvr2:27019,cfgsvr3:27019`. Kết nối tới mongos và thêm hai shard:

```
sh.addShard("shard1rs/shard1a:27018,shard1b:27018,shard1c:27018")
```

```
{
  shardAdded: 'shard1rs',
  ok: 1,
  ...
}
```

```
sh.addShard("shard2rs/shard2a:27018,shard2b:27018,shard2c:27018")
db.adminCommand({ listShards: 1 }).shards.map(s => ({ _id: s._id, host: s.host,
  state: s.state })))
```

```
[
  {
    _id: 'shard1rs',
    host: 'shard1rs/shard1a:27018,shard1b:27018,shard1c:27018',
    state: 1
  },
  {
    _id: 'shard2rs',
    host: 'shard2rs/shard2a:27018,shard2b:27018,shard2c:27018',
    state: 1
  }
]
```

Bước 4: Bật sharding cho cơ sở dữ liệu và tạo collection

Bật sharding cho QLThietBi với shard chính shard1rs; collection products sau đó được tạo bằng `db.createCollection` với cùng bộ kiểm tra `$jsonSchema` như mục 1.4.1:

```
sh.enableSharding("QLThietBi", "shard1rs")
```

Bước 5: Định nghĩa zone và khoảng giá trị shard key

Gán mỗi shard vào một zone và khai báo khoảng shard key cho từng zone; MinKey, MaxKey biểu diễn giá trị nhỏ nhất, lớn nhất. Khoảng có cận dưới đóng, cận trên mở:

```
sh.addShardToZone("shard1rs", "NSX_A_D");
sh.addShardToZone("shard2rs", "NSX_E_H")
```

```
sh.updateZoneKeyRange(
  "QLThietBi.products",
  { maker: MinKey, model: MinKey },
  { maker: "E", model: MinKey },
  "NSX_A_D"
)
sh.updateZoneKeyRange(
  "QLThietBi.products",
  { maker: "E", model: MinKey },
  { maker: MaxKey, model: MaxKey },
  "NSX_E_H"
)
```

Bước 6: Tạo chỉ mục shard key và phân mảnh collection

Tạo chỉ mục duy nhất trên shard key, sau đó phân mảnh collection với tham số `unique: true`. Vì zone đã được định nghĩa trước khi phân mảnh collection xong, MongoDB tạo sẵn các chunk theo zone và đặt chúng lên đúng shard:

```
db.products.createIndex({ maker: 1, model: 1 }, { unique: true,
  name: "uq_maker_model" })
sh.shardCollection("QLThietBi.products", { maker: 1, model: 1 }, true)
```

```
{
  collectionssharded: 'QLThietBi.products',
  ok: 1,
  ...
}
```

Bước 7: Nhập dữ liệu qua mongos

Dữ liệu được nhập qua mongos bằng đúng lệnh `insertMany` 30 document ở mục 1.4.1; mongos tự định tuyến từng document tới shard theo giá trị shard key. Collection makers được sinh bằng pipeline `$out` như Bước 4 mục 1.4.1. Kiểm tra số lượng và tổng giá sau khi nhập:

```
(( { products: db.products.countDocuments(), makers: db.makers.countDocuments() } ))
```

```
{ products: 30, makers: 8 }
```

```
db.products.aggregate([
  { $group: { _id: "$type", soLuong: { $sum: 1 }, tongGia: { $sum: "$price" } } },
  { $sort: { _id: 1 } }
])
```

```
[
  { _id: 'laptop', soLuong: 10, tongGia: 15830 },
  { _id: 'pc', soLuong: 13, tongGia: 0 },
  { _id: 'printer', soLuong: 7, tongGia: 1777 }
]
```

Số lượng (30 sản phẩm, 8 NSX) và tổng giá theo loại (15830, 1777) trùng khớp với dữ liệu nguồn và với kết quả trên Replica Set `rs0`.

3.3. Đánh giá kết quả triển khai

3.3.1. Kiểm tra sao chép dữ liệu

Trong Sharded Cluster, mỗi shard là một Replica Set nên dữ liệu của mỗi mảnh được sao chép trên ba nút. Kết nối trực tiếp tới từng nút của hai shard, đếm số

document bằng `countDocuments()` và tính giá trị băm bằng lệnh `dbHash` như mục 3.1.2; kết quả được tổng hợp trong Bảng 3.4.

Bảng 3.4. Kết quả kiểm tra sao chép dữ liệu trên các nút của hai shard

Nút	Vai trò	products	makers	md5 products
shard1a	Primary	16	8	379c70fa218153278358b59086730c30
shard1b	Secondary	16	8	379c70fa218153278358b59086730c30
shard1c	Secondary	16	8	379c70fa218153278358b59086730c30
shard2a	Primary	14	—	883b1a69168aa73617dfbe9da74e85a8
shard2b	Secondary	14	—	883b1a69168aa73617dfbe9da74e85a8
shard2c	Secondary	14	—	883b1a69168aa73617dfbe9da74e85a8

Ba nút của `shard1rs` cùng lưu 16 sản phẩm và 8 NSX với cùng giá trị băm; ba nút của `shard2rs` cùng lưu 14 sản phẩm với cùng giá trị băm. Giá trị băm của `makers` trên `shard1rs` (`d5370fd06a542c1d77000b1c90b07829`) trùng với giá trị trên Replica Set `rs0` ở Bảng 3.1 – dữ liệu NSX ở hai môi trường giống hệt nhau.

Kiểm thử sự cố nút Primary trong một shard

Để đánh giá tính sẵn sàng của cụm, container `shard1a` (Primary của `shard1rs`) bị dừng đột ngột. Kết quả (Hình 3.7) cho thấy: truy vấn qua mongos vẫn trả về đủ 30 sản phẩm và 6 sản phẩm của NSX A (dữ liệu thuộc `shard1rs`); thao tác cập nhật vào dữ liệu của `shard1rs` vẫn thành công; `shard1b` đã được bầu làm Primary mới của `shard1rs`.

```

Terminal & mongosh — dừng nút Primary của shard1rs

$ docker stop shard1a && sleep 20 && docker ps -a --filter name=shard1a --format "{{.Names}}:
  {{.Status}}"
shard1a
shard1a: Exited (137) 21 seconds ago

[direct: mongos] QLThietBi> ({ tongSanPham: db.products.countDocuments(), sanPhamNSX_A:
  db.products.countDocuments({ maker: "A" }) })
{ tongSanPham: 30, sanPhamNSX_A: 6 }

[direct: mongos] QLThietBi> db.products.updateOne({ maker: "A", model: 2004 }, { $set: { price: 1199 }
  })
{
  ...
  matchedCount: 1,
  modifiedCount: 1,
  ...
}

```

Hình 3.7. Dừng nút Primary `shard1a` – cụm vẫn phục vụ đọc, ghi qua mongos

Trạng thái của `shard1rs` khi `shard1a` dừng (kết nối tới `shard1b`):

```
rs.status().members.map(m => ({ name: m.name, stateStr: m.stateStr, health: m.health })))
```

```
[
  {
    name: 'shard1a:27018',
    stateStr: '(not reachable/healthy)',
    health: 0
  },
  { name: 'shard1b:27018', stateStr: 'PRIMARY', health: 1 },
  { name: 'shard1c:27018', stateStr: 'SECONDARY', health: 1 }
]
```

Giá trị cập nhật thử nghiệm (price = 1199) được kiểm tra qua mongos rồi hoàn tác về giá gốc 1150 khi shard1a vẫn đang dừng.

Khởi động lại shard1a (docker start shard1a), kiểm tra trạng thái và dữ liệu trên chính nút này:

```
rs.status().members.map(m => ({ name: m.name, stateStr: m.stateStr, health: m.health })))
```

```
[
  { name: 'shard1a:27018', stateStr: 'PRIMARY', health: 1 },
  { name: 'shard1b:27018', stateStr: 'SECONDARY', health: 1 },
  { name: 'shard1c:27018', stateStr: 'SECONDARY', health: 1 }
]
```

```
db.products.findOne({ maker: "A", model: 2004 }, { _id: 0, maker: 1, model: 1,
  price: 1 })
```

```
{ maker: 'A', model: 2004, price: 1150 }
```

shard1a đã trở lại vai trò Primary và có giá trị price = 1150, tức là nút đã áp dụng đầy đủ hai thao tác cập nhật diễn ra trong thời gian nó dừng. Cơ chế sao chép bên trong từng shard bảo đảm Sharded Cluster không có điểm lỗi đơn ở tầng lưu trữ dữ liệu.

3.3.2. Kiểm tra phân bố dữ liệu trên các Shard

a) Phân bố document và chunk

Phương thức getShardDistribution() thống kê dữ liệu của collection trên từng shard (Hình 3.8): shard1rs lưu 16 document (53,33%), shard2rs lưu 14 document (46,66%), mỗi shard một chunk.

```

○ ○ ○ mongosh — mongos:27017 (Sharded Cluster)

[direct: mongos] QLThietBi> db.products.getShardDistribution()
Shard shard2rs at shard2rs/shard2a:27018,shard2b:27018,shard2c:27018
{
  data: '1KiB',
  docs: 14,
  chunks: 1,
  ...
}
---
Shard shard1rs at shard1rs/shard1a:27018,shard1b:27018,shard1c:27018
{
  data: '1KiB',
  docs: 16,
  chunks: 1,
  ...
}
---
Totals
{
  data: '2KiB',
  docs: 30,
  chunks: 2,
  'Shard shard2rs': [
    '53.31 % data',
    '46.66 % docs in cluster',
    '109B avg obj size on shard'
  ],
  'Shard shard1rs': [
    '46.68 % data',
    '53.33 % docs in cluster',
    '83B avg obj size on shard'
  ]
}

```

Hình 3.8. Kết quả getShardDistribution() của collection products

Stage \$shardedDataDistribution cho biết thêm số document mồ côi (orphaned) – document nằm trên shard nhưng không thuộc chunk của shard đó:

```

db.getSiblingDB("admin").aggregate([
  { $shardedDataDistribution: {} },
  { $match: { ns: "QLThietBi.products" } }
]).toArray()[0].shards

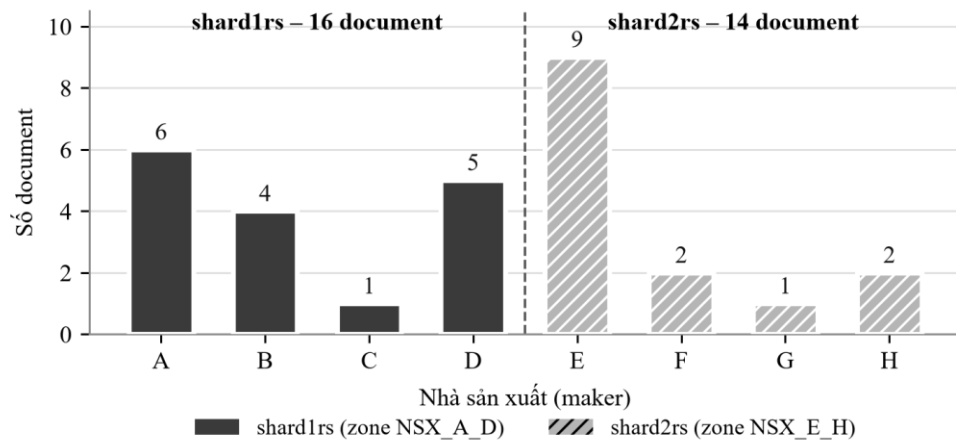
```

```

[
  {
    shardName: 'shard1rs',
    numOrphanedDocs: 0,
    numOwnedDocuments: 16,
    ...
  },
  {
    shardName: 'shard2rs',
    numOrphanedDocs: 0,
    numOwnedDocuments: 14,
    ...
  }
]

```

Truy vấn collection config.chunks trên config server cho thấy hai chunk có biên trùng khớp với hai zone đã khai báo: chunk [{maker: MinKey}, {maker: "E"}] trên shard1rs và chunk [{maker: "E"}, {maker: MaxKey}] trên shard2rs; số document mồ côi bằng 0. Phân bố chi tiết theo NSX được trực quan hóa trong Hình 3.9.



Hình 3.9. Phân bố document của collection products theo nhà sản xuất trên hai shard

b) Kiểm tra định tuyến truy vấn

Phương thức `explain()` cho biết kế hoạch thực thi mà mongos lựa chọn. Hình 3.10 và Bảng 3.5 cho thấy: truy vấn có điều kiện trên tiền tố shard key (maker) được định tuyến tới **một shard duy nhất** (SINGLE_SHARD); truy vấn không chứa shard key (type, hoặc chỉ model) phải gửi tới **mọi shard** rồi hợp nhất kết quả (SHARD_MERGE).

```

○○○ mongosh — mongos:27017 (Sharded Cluster)

[direct: mongos] QLThietBi> (e => ({ stage: ..., shards: ..., nReturned: ... }))(
  db.products.find({ maker: "A" }).explain("executionStats")
)
{
  stage: 'SINGLE_SHARD',
  shards: [ { shard: 'shard1rs', plan: 'FETCH <- SHARDING_FILTER' } ],
  nReturned: 6
}

[direct: mongos] QLThietBi> (e => ({ stage: ..., shards: ..., nReturned: ..., nReturnedPerShard: ...
  }))(
  db.products.find({ type: "laptop" }).explain("executionStats")
)
{
  stage: 'SHARD_MERGE',
  shards: [ 'shard1rs', 'shard2rs' ],
  nReturned: 10,
  nReturnedPerShard: [ { shard: 'shard1rs', n: 4 }, { shard: 'shard2rs', n: 6 } ]
}

```

Hình 3.10. Kế hoạch thực thi của truy vấn có và không có shard key

Bảng 3.5. Kết quả định tuyến truy vấn trên Sharded Cluster

Truy vấn	Kế hoạch	Shard tham gia	Số kết quả
{ maker: "A" }	SINGLE_SHARD	shard1rs	6
{ maker: "E", model: 2001 }	SINGLE_SHARD	shard2rs	1
{ maker: { \$in: ["F", "G", "H"] } }	SINGLE_SHARD	shard2rs	5
{ type: "laptop" }	SHARD_MERGE	shard1rs (4), shard2rs (6)	10
{ model: 2001 }	SHARD_MERGE	shard1rs, shard2rs	1

Truy vấn tổng hợp đếm sản phẩm theo NSX qua mongos được thực hiện song song trên hai shard, hợp nhất tại mongos và cho kết quả trùng với Bảng 1.4.

c) So sánh với phương án khóa băm

Để đối chiếu, một bản sao của dữ liệu được nạp vào collection thử nghiệm products_hashed phân mảnh theo khóa băm { model: "hashed" }, sau đó kiểm tra phân bố bằng \$shardedDataDistribution và kế hoạch thực thi bằng explain() như các mục trên (collection được xóa sau khi so sánh). Kết quả so sánh được tổng hợp trong Bảng 3.6.

```
sh.shardCollection("QLThietBi.products_hashed", { model: "hashed" })
db.products.aggregate([
  { $project: { _id: 0 } },
  { $merge: { into: "products_hashed" } }
])
```

Bảng 3.6. So sánh hai phương án shard key qua thực nghiệm

Tiêu chí	{ maker: 1, model: 1 } + zone	{ model: "hashed" }
Phân bố document (shard1rs / shard2rs)	16 / 14	14 / 16
Kiểm soát vị trí dữ liệu	Tường minh theo NSX (zone)	Ngẫu nhiên theo giá trị băm
Truy vấn { maker: "A" }	SINGLE_SHARD	SHARD_MERGE (2 shard)
Truy vấn { model: 2001 }	SHARD_MERGE (2 shard)	SINGLE_SHARD
Truy vấn khoảng theo NSX	Định tuyến được	Phải gửi mọi shard
Phân tán ghi khi thêm model mới	Theo NSX của sản phẩm	Đều trên các shard

Cả hai phương án đều phân bố dữ liệu tương đối cân bằng. Khóa băm phân tán ghi đều nhưng mọi truy vấn theo NSX phải gửi tới tất cả shard; với mẫu truy cập của bài toán (TC2, TC5 thường xuyên), khóa ghép { maker: 1, model: 1 } kết hợp zone là lựa chọn phù hợp hơn.

d) Đánh giá tổng hợp

Bảng 3.7. Đánh giá kết quả quản trị dữ liệu trên MongoDB

Tiêu chí	Kết quả	Minh chứng
Toàn vẹn dữ liệu	30 sản phẩm, 8 NSX; tổng giá 15830 / 1777 trên cả hai môi trường	verify_restore, verify_sum
Đồng bộ giữa các nút	Giá trị bấm trùng nhau trên mọi nút của rs0, shard1rs, shard2rs	Bảng 3.1, Bảng 3.4
Tính sẵn sàng	Tự bầu Primary mới khi Primary dừng; đọc ghi không gián đoạn; nút phục hồi tự bắt kịp	Hình 3.4, Hình 3.7
Phân bố dữ liệu	16 / 14 document đúng theo zone; 0 document mồ côi	Hình 3.8
Định tuyến truy vấn	Truy vấn theo maker đến 1 shard; truy vấn khác hợp nhất từ 2 shard	Bảng 3.5

Kết luận Chương 3

Chương 3 đã triển khai thành công QLThietBi trên môi trường phân tán. Replica Set rs0 ba nút sao chép dữ liệu qua oplog với độ trễ 0 giây, giá trị bấm trên ba nút trùng khớp; khi Primary gặp sự cố, cụm tự bầu Primary mới và nút phục hồi tự bắt kịp dữ liệu. Khóa phân mảnh { maker: 1, model: 1 } kết hợp zone theo NSX được lựa chọn qua phân tích lực lượng, tần suất, tính đơn điệu và khả năng định tuyến, tương ứng phân mảnh ngang nguyên thủy theo maker. Sharded Cluster phân bố dữ liệu đúng thiết kế (16 / 14 document), định tuyến truy vấn theo NSX tới một shard và vẫn phục vụ khi một nút trong shard gặp sự cố.

KẾT LUẬN

Kết quả đạt được

Tiểu luận đã hoàn thành các mục tiêu nghiên cứu đề ra đối với bài toán xây dựng và quản trị cơ sở dữ liệu quản lý thiết bị máy tính trên MongoDB:

- Phân tích bài toán, dữ liệu nguồn và các mẫu truy cập; lựa chọn mô hình kết hợp gồm products nhúng cấu hình vào specs và makers lưu mảng tham chiếu, có ràng buộc \$jsonSchema và chỉ mục duy nhất.
- Nhập đầy đủ, chính xác 30 sản phẩm theo dữ liệu nguồn; thực hiện tạo lập, tìm kiếm, cập nhật (có hoàn tác) trên document, embedded document và mảng; hiện thực 16 yêu cầu khai thác bằng find, Aggregation Pipeline và \$lookup với kết quả được đối chiếu chéo.
- Cấu hình Replica Set ba nút, kiểm chứng đồng bộ (oplog, độ trễ, giá trị băm) và chuyển đổi dự phòng; lựa chọn shard key, xây dựng Sharded Cluster hai shard với zone theo NSX, kiểm tra phân bố và định tuyến truy vấn, so sánh với phương án khóa băm.

Kết quả cho thấy MongoDB phù hợp với dữ liệu danh mục sản phẩm không đồng nhất: mô hình document biểu diễn tự nhiên các loại sản phẩm, Aggregation Pipeline đáp ứng tốt yêu cầu thống kê, Replica Set và Sharding cung cấp tính sẵn sàng và khả năng mở rộng mà không cần thay đổi ứng dụng.

Hạn chế

- Dữ liệu nhỏ (30 sản phẩm, khoảng 3 KB) nên chưa đánh giá được hiệu năng, quá trình tự chia chunk và hoạt động của balancer ở quy mô lớn.
- Toàn bộ 13 tiến trình máy chủ chạy trên một máy tính bằng Docker, chưa phản ánh độ trễ mạng và sự cố phần cứng thực tế; môi trường chưa bật xác thực, phân quyền và mã hóa TLS.

Hướng phát triển

- Sinh dữ liệu mô phỏng quy mô lớn để đánh giá hiệu năng, quan sát chia chunk, cân bằng dữ liệu; triển khai cụm trên nhiều máy chủ hoặc MongoDB Atlas với cơ chế bảo mật và giám sát.
- Tối ưu truy vấn bằng chỉ mục; dùng giao tác đa tài liệu để đồng bộ products – makers khi bổ sung sản phẩm (trên collection phân mảnh, tính duy nhất của model hiện dựa vào chỉ mục ghép { maker, model }).

TÀI LIỆU THAM KHẢO

Tài liệu tiếng Việt

- [1]. Sở Bưu chính Viễn thông thành phố Hà Nội (2005), *Giáo trình Cơ sở dữ liệu*, Sở Bưu chính Viễn thông thành phố Hà Nội, Hà Nội.
- [2]. *Oracle cơ bản – SQL và PL/SQL*, Cơ sở dữ liệu thực hành.

Tài liệu tiếng Anh

- [3]. Chellappan, Subhashini and Ganesan, Dharanitharan (2020), *MongoDB Recipes: With Data Modeling and Query Building Strategies*, Apress, Berkeley, CA.
- [4]. Chodorow, Kristina (2013), *MongoDB: The Definitive Guide*, 2nd edition, O'Reilly Media, Sebastopol, CA.
- [5]. Elmasri, Ramez and Navathe, Shamkant B. (2016), *Fundamentals of Database Systems*, 7th edition, Pearson, Boston.
- [6]. Garcia-Molina, Hector, Ullman, Jeffrey D. and Widom, Jennifer (2009), *Database Systems: The Complete Book*, Second Edition, Pearson Prentice Hall, Upper Saddle River, New Jersey.
- [7]. MongoDB, Inc. (2025), *MongoDB Manual – Version 8.0*, MongoDB, Inc., New York. Truy cập tại: <https://www.mongodb.com/docs/manual/>.